Cryptography

From Modes of Operation to Authenticated Encryption

Arkady Yerukhimovich

September 28, 2026

Announcements

- HW4 is due Monday, October 5
- Start thinking about your project topic and team
- Midterm exam in class on Monday, October 19

Outline

1. Modes of Operation
2. Padding-Oracle Attack on CBC Mode
3. Chosen-Ciphertext Attack (CCA) Security
4. Message Authentication Codes
5. Authenticated Encryption

Where We Left Off

- PRF+OTP is CPA-secure
- Can encrypt many n-bit messages with a single n-bit key

Where We Left Off

- PRF+OTP is CPA-secure
- Can encrypt many n-bit messages with a single n-bit key

But we want to encrypt messages with $>> n$ bits

Where We Left Off

- PRF+OTP is CPA-secure
- Can encrypt many n-bit messages with a single n-bit key

But we want to encrypt messages with $>> n$ bits

- A mode of operation specifies how to use a PRF to encrypt long messages

Where We Left Off

- PRF+OTP is CPA-secure
- Can encrypt many n-bit messages with a single n-bit key

But we want to encrypt messages with $>> n$ bits

- A mode of operation specifies how to use a PRF to encrypt long messages
- Combine PRF, randomness, and message blocks in a clever way

Where We Left Off

- PRF+OTP is CPA-secure
- Can encrypt many n-bit messages with a single n-bit key

But we want to encrypt messages with $>> n$ bits

- A mode of operation specifies how to use a PRF to encrypt long messages
- Combine PRF, randomness, and message blocks in a clever way
- But, we must be careful: some modes (e.g., ECB) are insecure!

Modes of Operation

A block cipher is an efficiently invertible keyed permutation:

$$F_k : \{0,1\}^n \longrightarrow \{0,1\}^n, \qquad \text{with efficient inverse } F_k^{-1}.$$

We model it as a secure pseudorandom permutation.

- Split a long message into blocks $m_1, \ldots, m_\ell$.
- A mode specifies how to combine block-cipher calls.

Modes of Operations: Important Properties

- Ciphertext length – what is the increase between $|c|$ and $|m|$

Modes of Operations: Important Properties

- Ciphertext length – what is the increase between $|c|$ and $|m|$
- Pre-computation – can some part of encryption procedure be computed without m

Modes of Operations: Important Properties

- Ciphertext length – what is the increase between $|c|$ and $|m|$
- Pre-computation – can some part of encryption procedure be computed without m
- Encryption/Decryption parallelism - can we encrypt/decrypt only part of m, or encrypt blocks in parallel

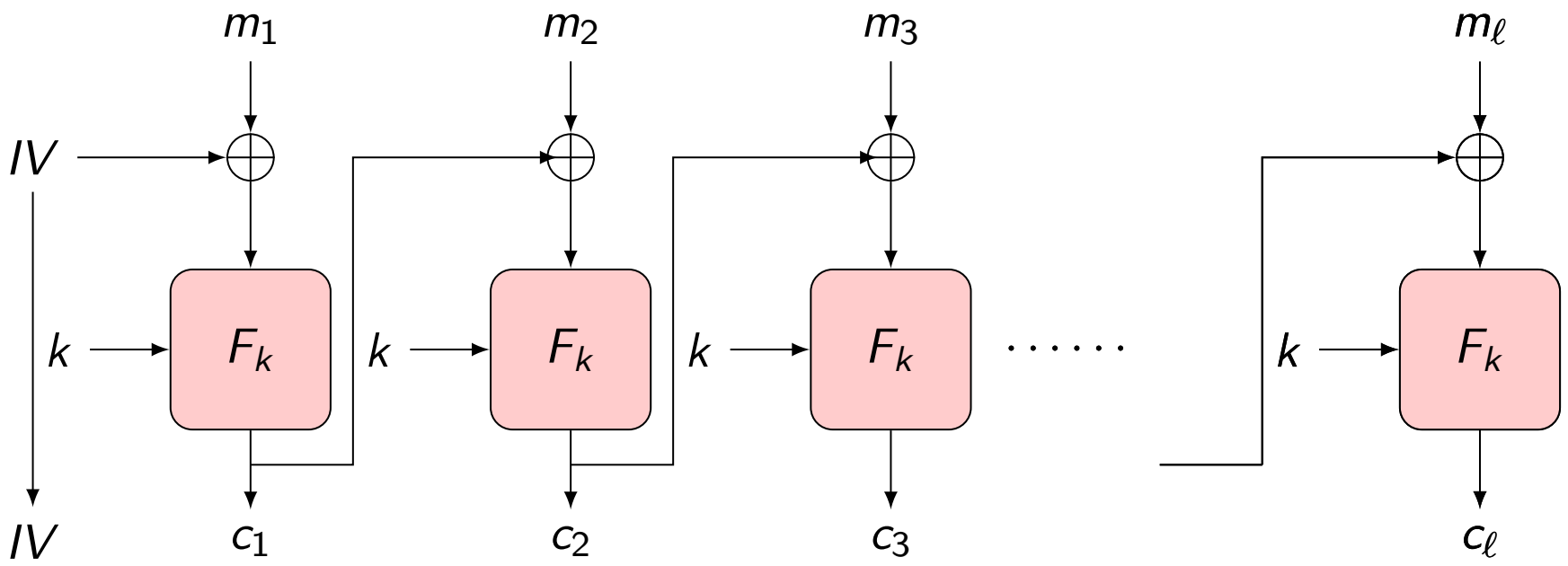
Modes of Operations: Important Properties

- Ciphertext length – what is the increase between $|c|$ and $|m|$
- Pre-computation – can some part of encryption procedure be computed without m
- Encryption/Decryption parallelism - can we encrypt/decrypt only part of m, or encrypt blocks in parallel
- PRF Inverse – do we need to be able to invert the PRF (i.e., do we need a PRP)

Modes of Operations: Important Properties

- Ciphertext length – what is the increase between $|c|$ and $|m|$
- Pre-computation – can some part of encryption procedure be computed without m
- Encryption/Decryption parallelism - can we encrypt/decrypt only part of m, or encrypt blocks in parallel
- PRF Inverse – do we need to be able to invert the PRF (i.e., do we need a PRP)
- Security - want at least CPA-security

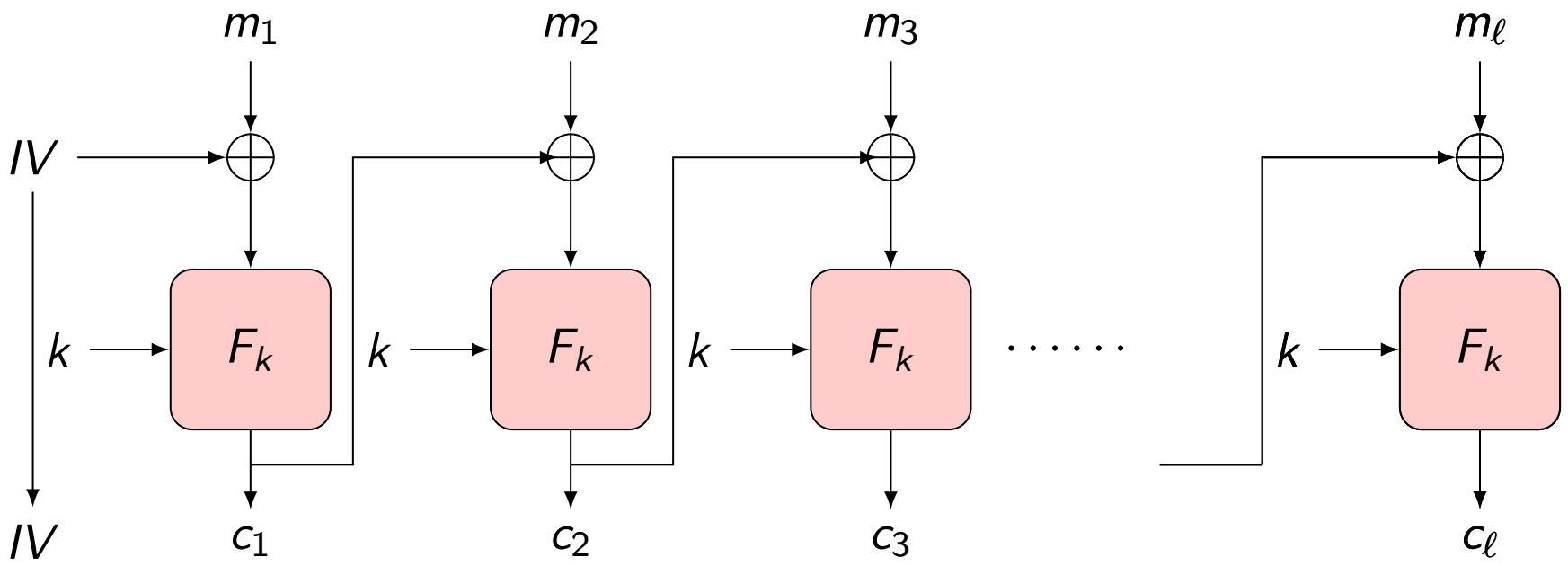
CBC: Cipher Block Chaining



Encrypt: Choose a fresh uniform IV; set $c_0 = IV$.

$$c_i = F_k(m_i \oplus c_{i-1}), \qquad c = (IV, c_1, \ldots, c_\ell).$$

CBC: Cipher Block Chaining

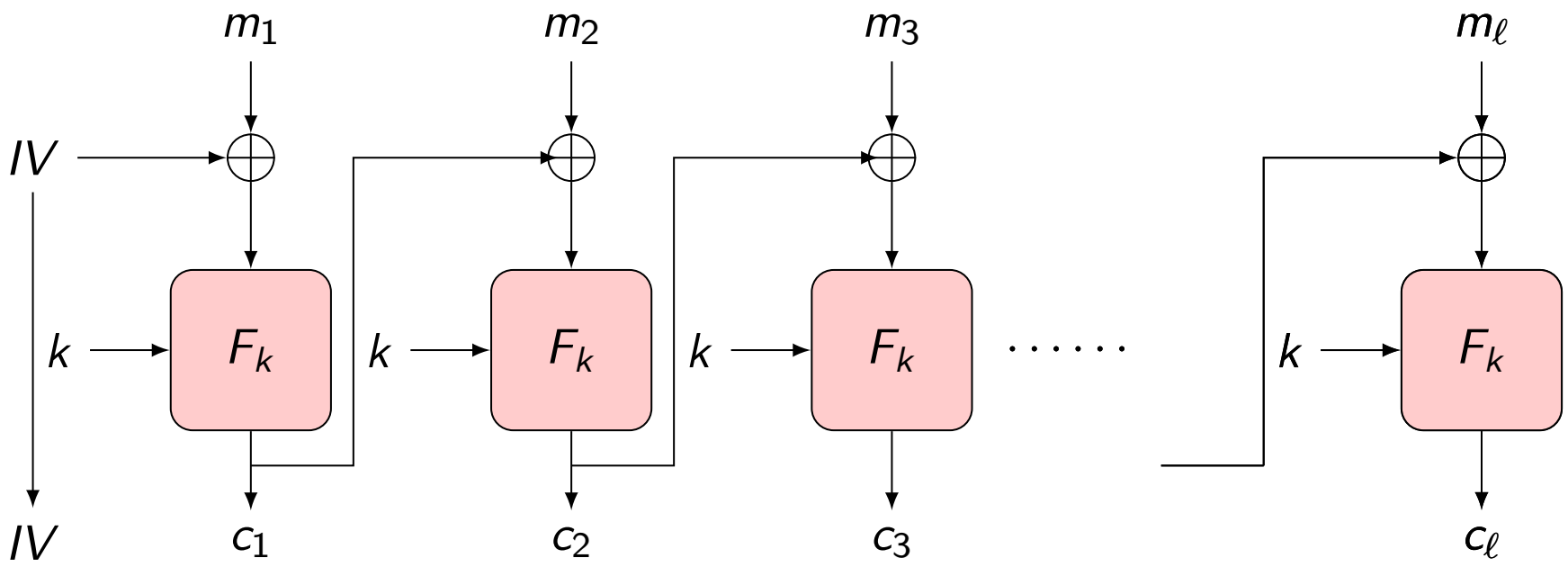


Encrypt: Choose a fresh uniform IV; set $c_0 = IV$.

$$c_i = F_k(m_i \oplus c_{i-1}), \qquad c = (IV, c_1, \ldots, c_\ell).$$

The public IV must be unpredictable when the plaintext is chosen.

CBC: Cipher Block Chaining



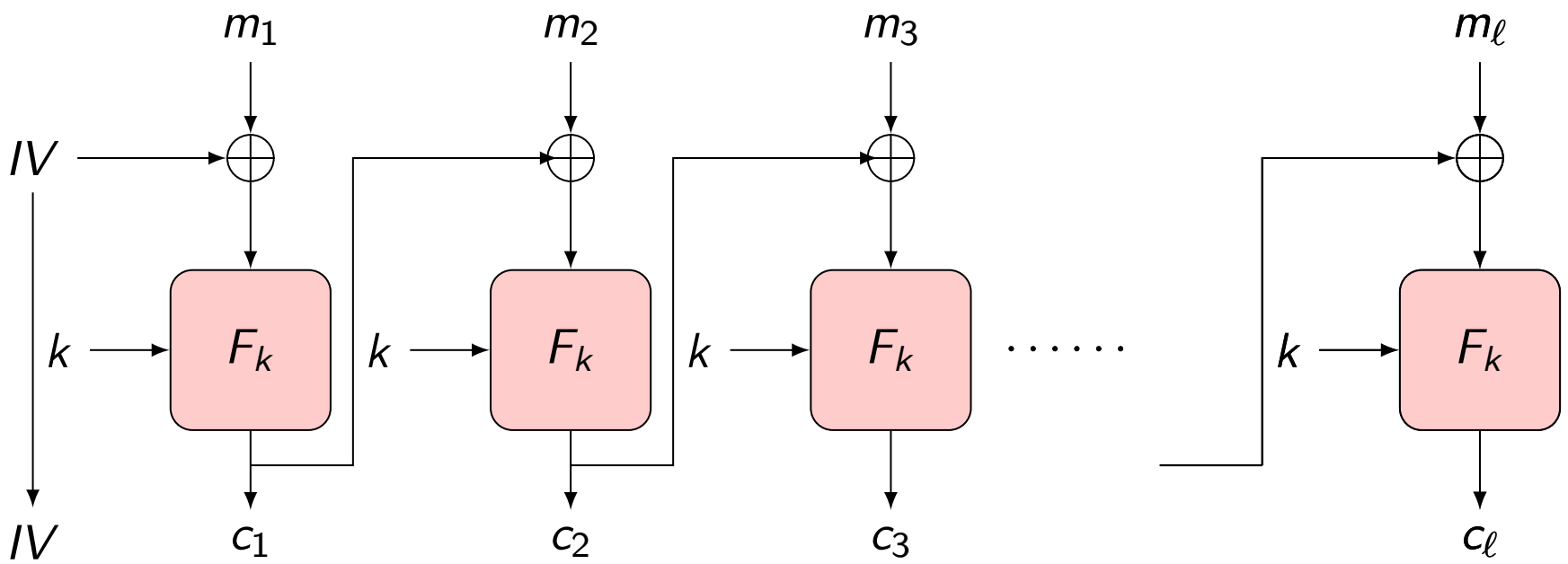
Encrypt: Choose a fresh uniform IV; set $c_0 = IV$.

$$c_i = F_k(m_i \oplus c_{i-1}), \qquad c = (IV, c_1, \ldots, c_\ell).$$

The public IV must be unpredictable when the plaintext is chosen.

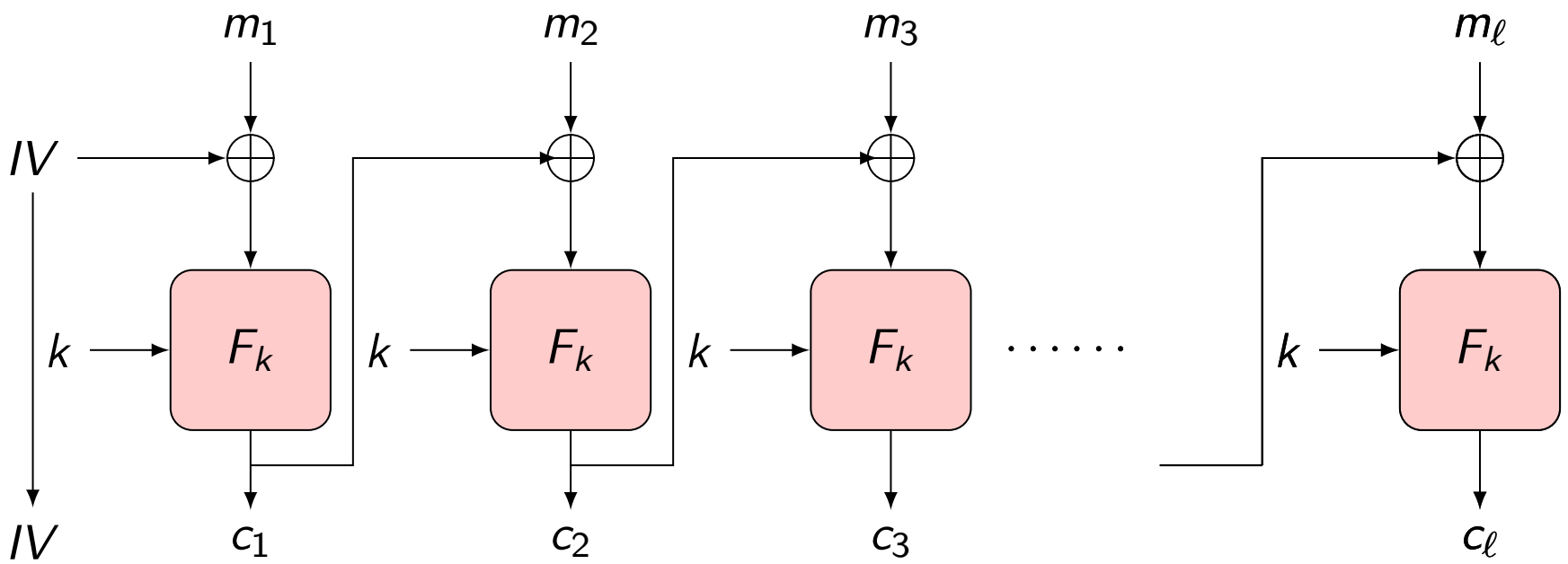
Decrypt: $m_i = F_k^{-1}(c_i) \oplus c_{i-1}$.

CBC Mode Properties



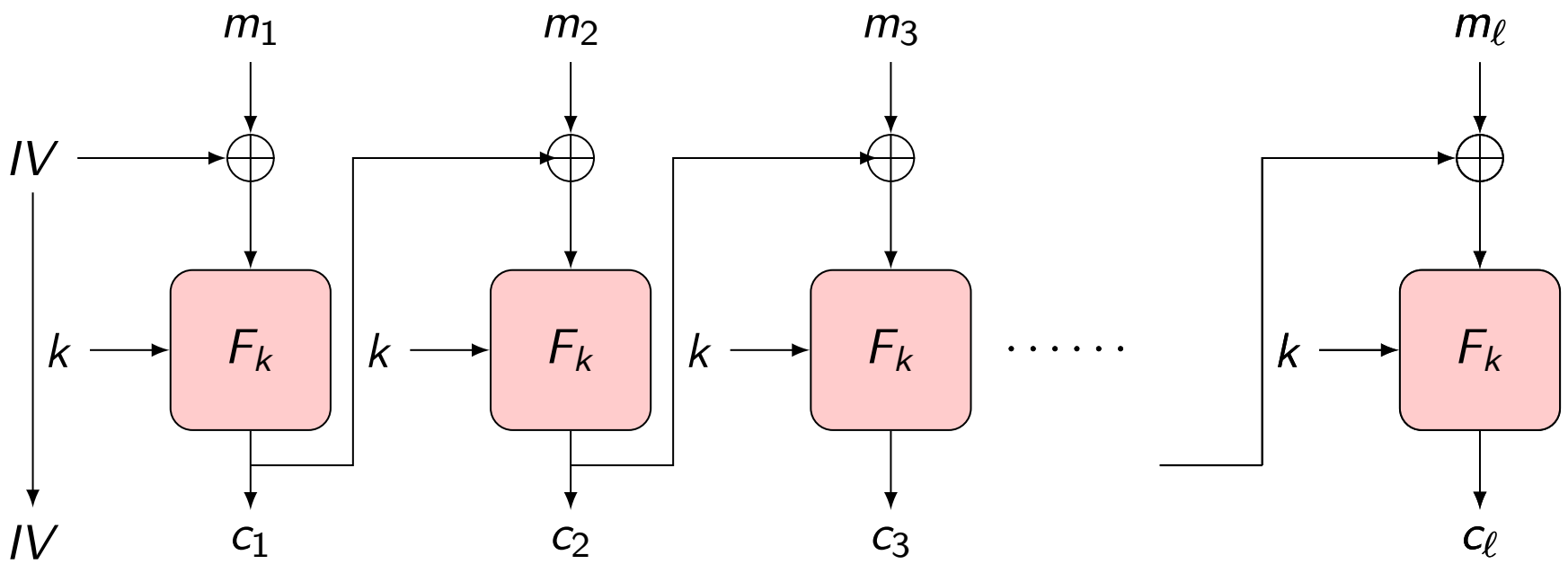
- Ciphertext length: $|c| = |m| + 1$ (adds one block) (assuming $|m|$ is a multiple of n)

CBC Mode Properties



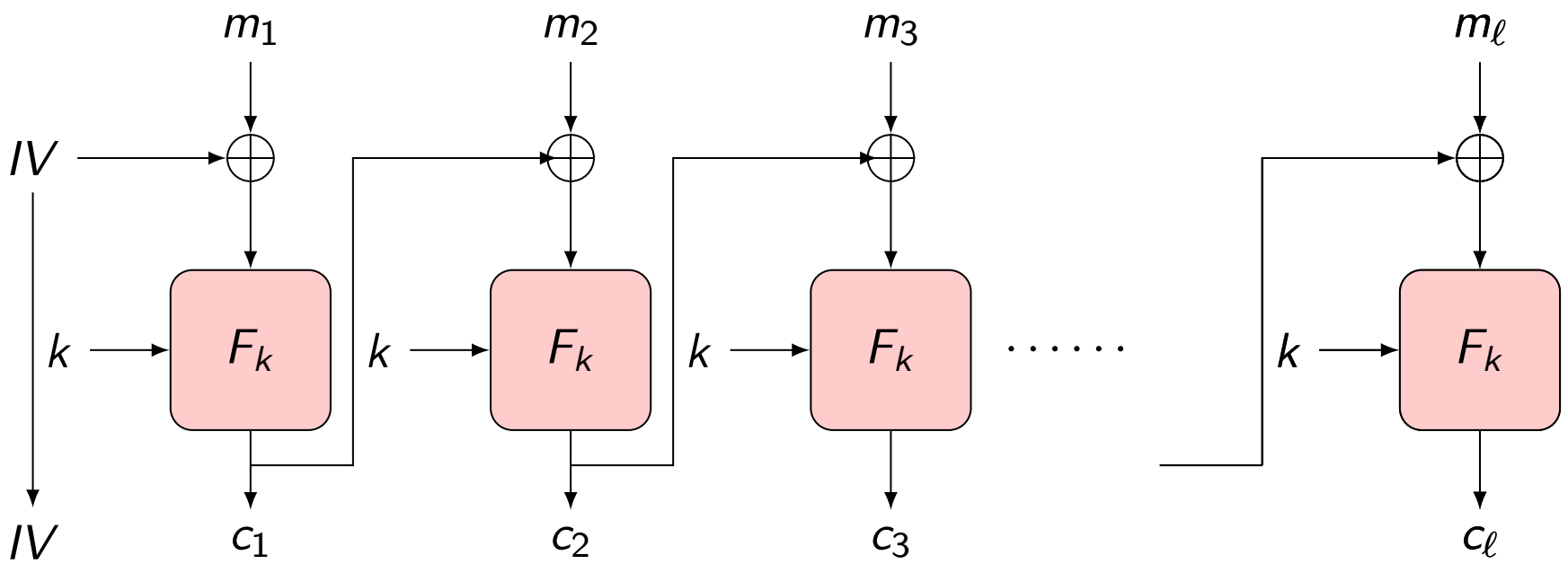
- Ciphertext length: $|c| = |m| + 1$ (adds one block) (assuming $|m|$ is a multiple of n)
- Pre-computation: No

CBC Mode Properties



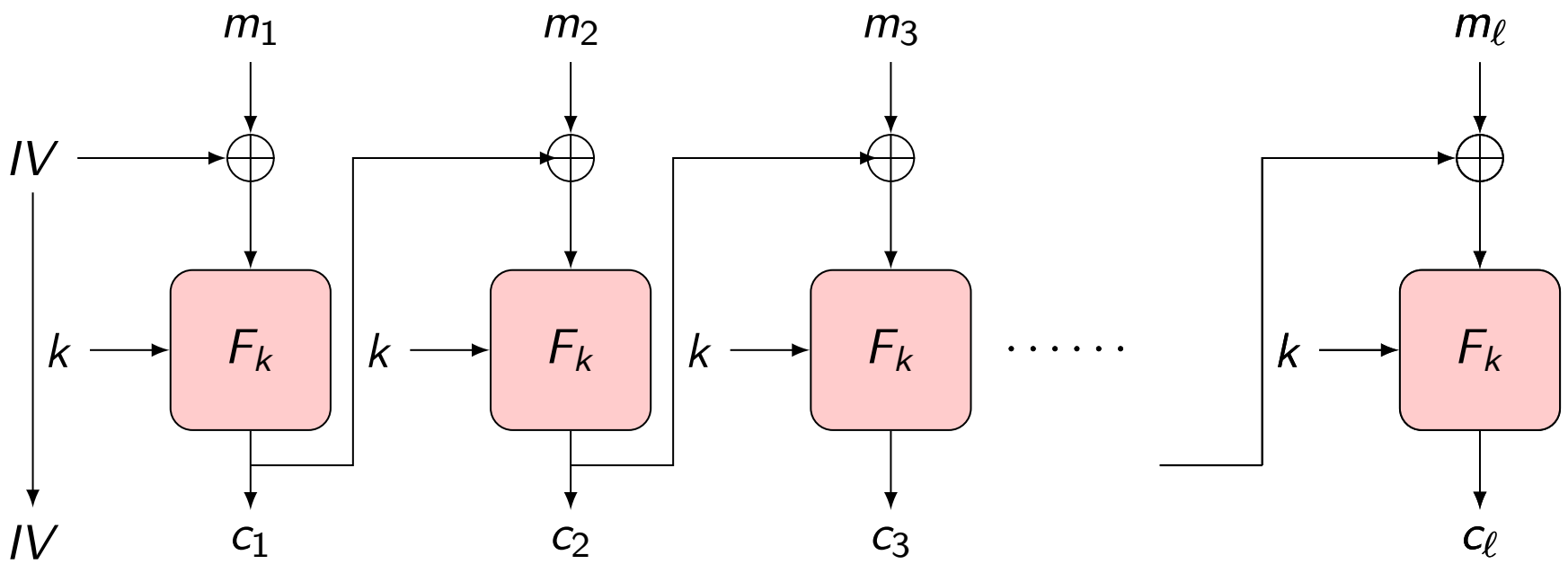
- Ciphertext length: $|c| = |m| + 1$ (adds one block) (assuming $|m|$ is a multiple of n)
- Pre-computation: No
- Encryption/Decryption parallelism: Cannot parallelize encryption, but can parallelize decryption

CBC Mode Properties



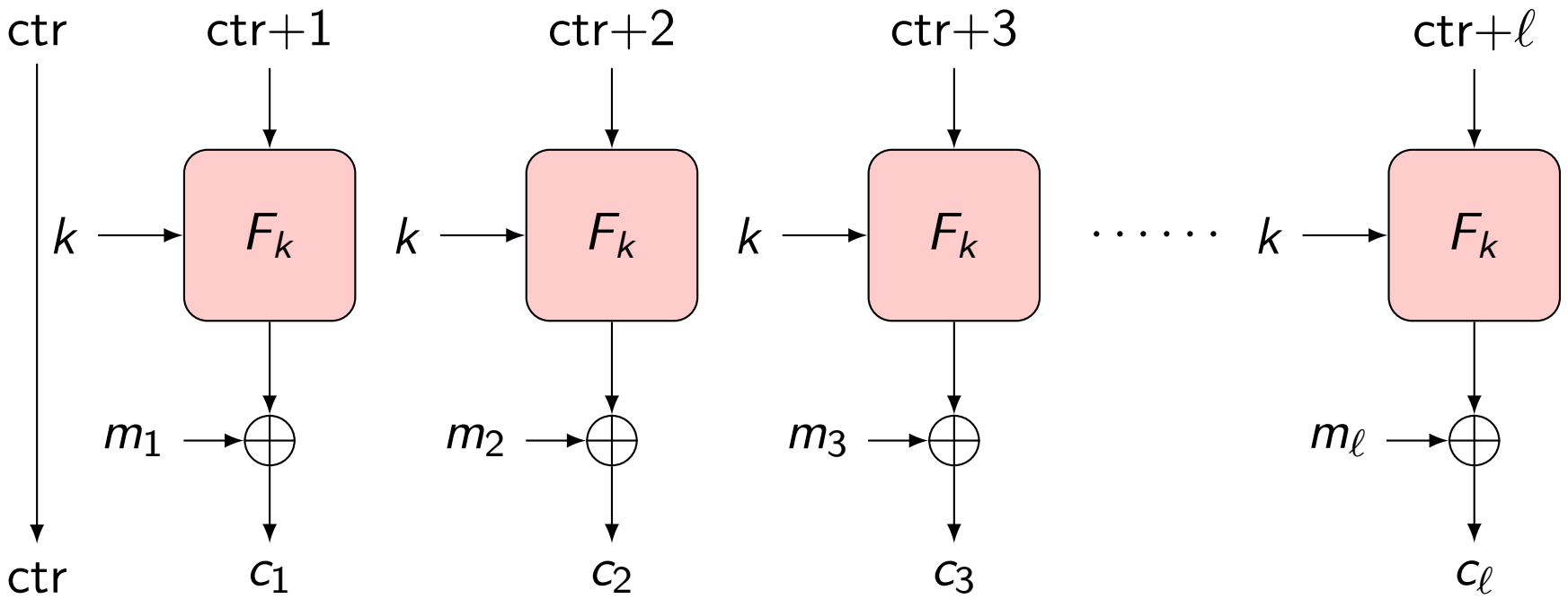
- Ciphertext length: $|c| = |m| + 1$ (adds one block) (assuming $|m|$ is a multiple of n)
- Pre-computation: No
- Encryption/Decryption parallelism: Cannot parallelize encryption, but can parallelize decryption
- PRF Inverse: Yes

CBC Mode Properties



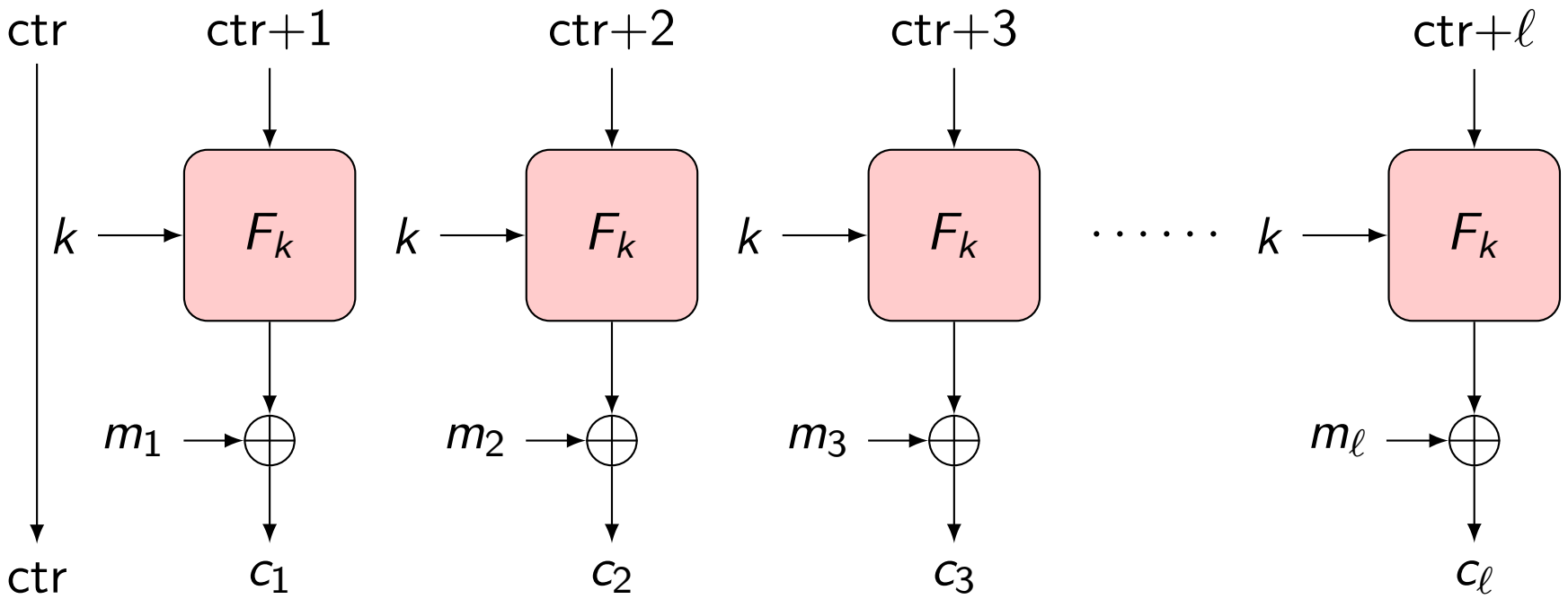
- Ciphertext length: $|c| = |m| + 1$ (adds one block) (assuming $|m|$ is a multiple of n)
- Pre-computation: No
- Encryption/Decryption parallelism: Cannot parallelize encryption, but can parallelize decryption
- PRF Inverse: Yes
- Security: CPA-secure if IV is unpredictable

CTR: Counter Mode



$$c_i = m_i \oplus F_k(ctr + i), \qquad m_i = c_i \oplus F_k(ctr + i).$$

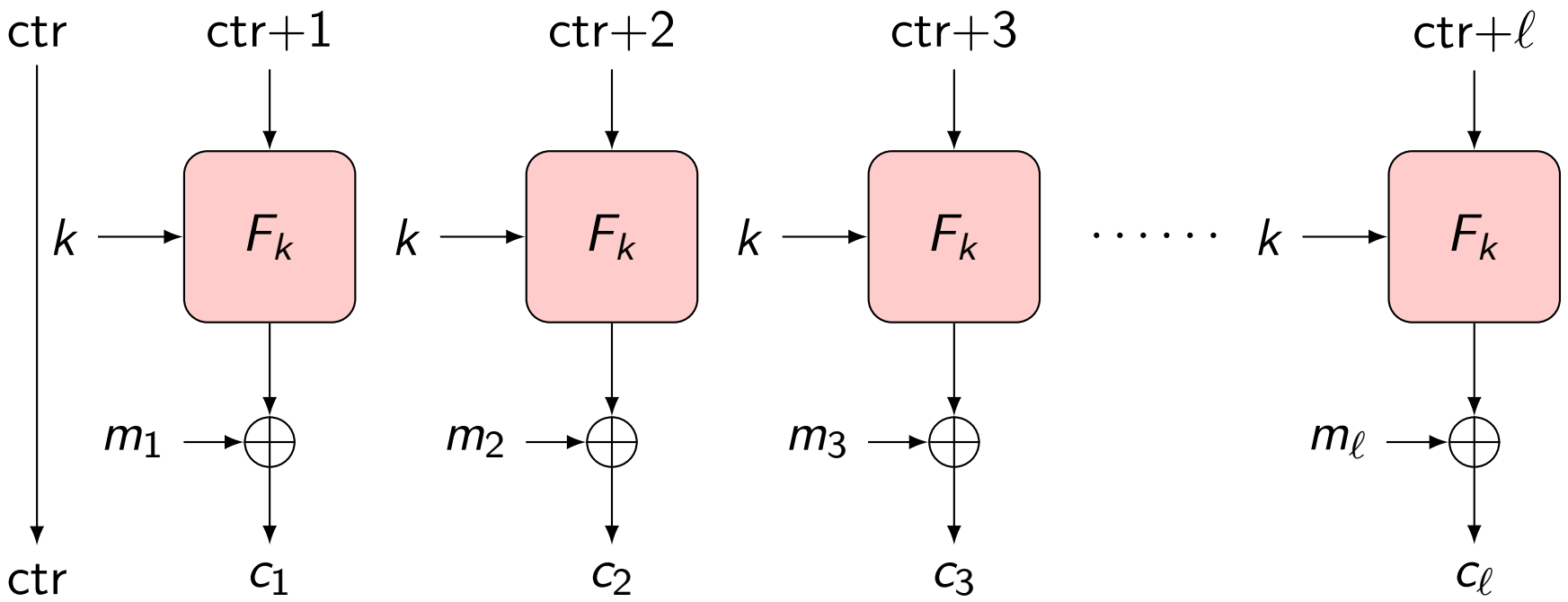
CTR: Counter Mode



$$c_i = m_i \oplus F_k(ctr + i), \qquad m_i = c_i \oplus F_k(ctr + i).$$

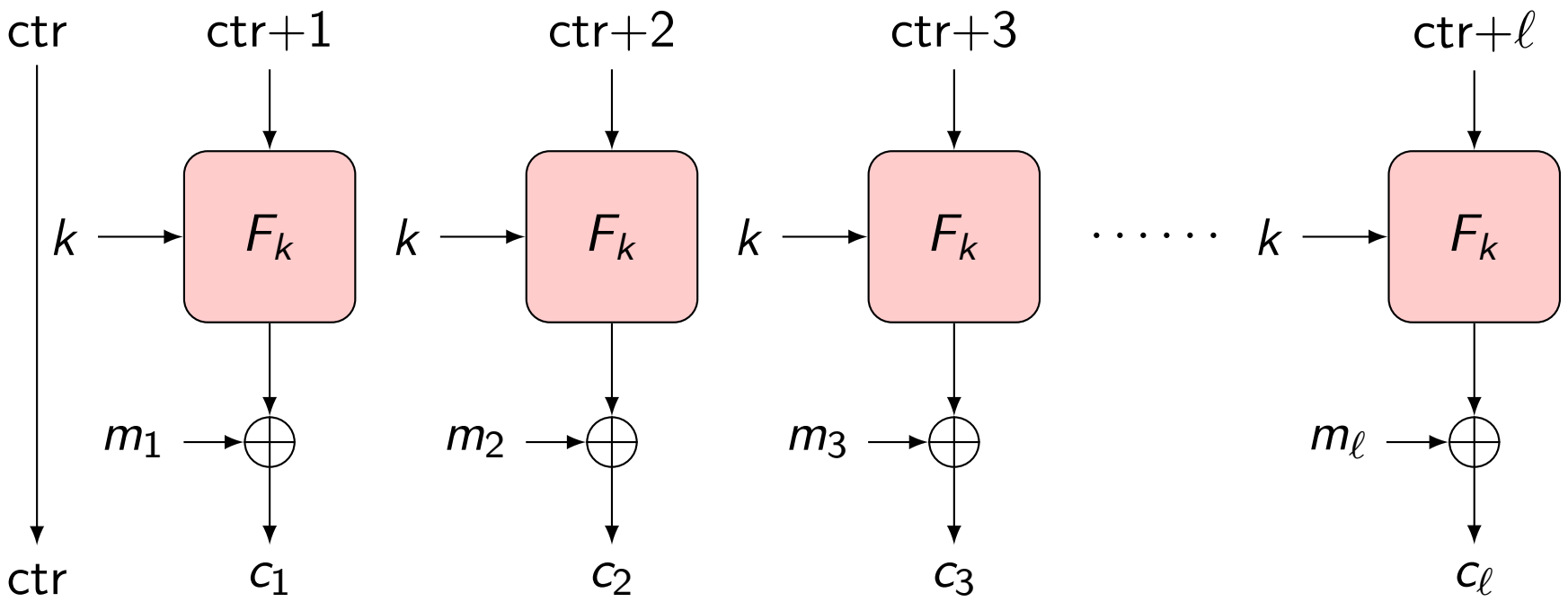
Transmit *ctr*. Never reuse a counter input under the same key.

CTR Mode Properties



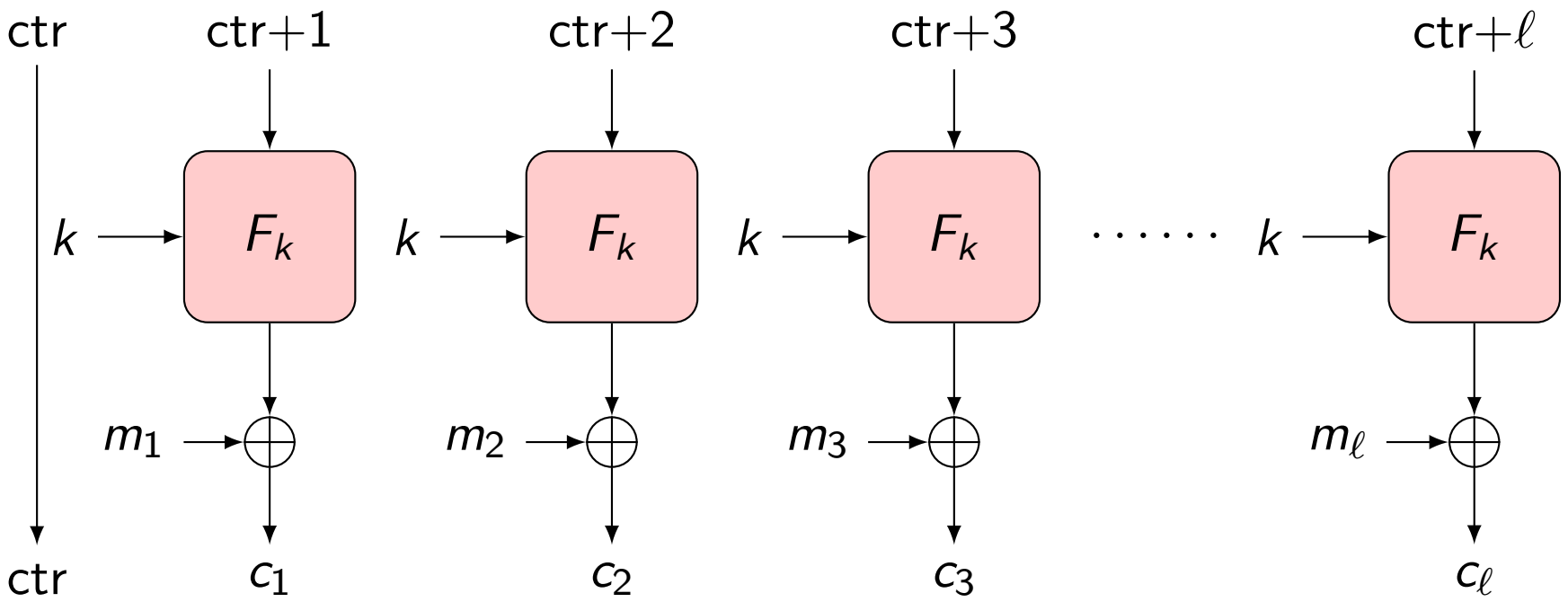
- Ciphertext length:

CTR Mode Properties



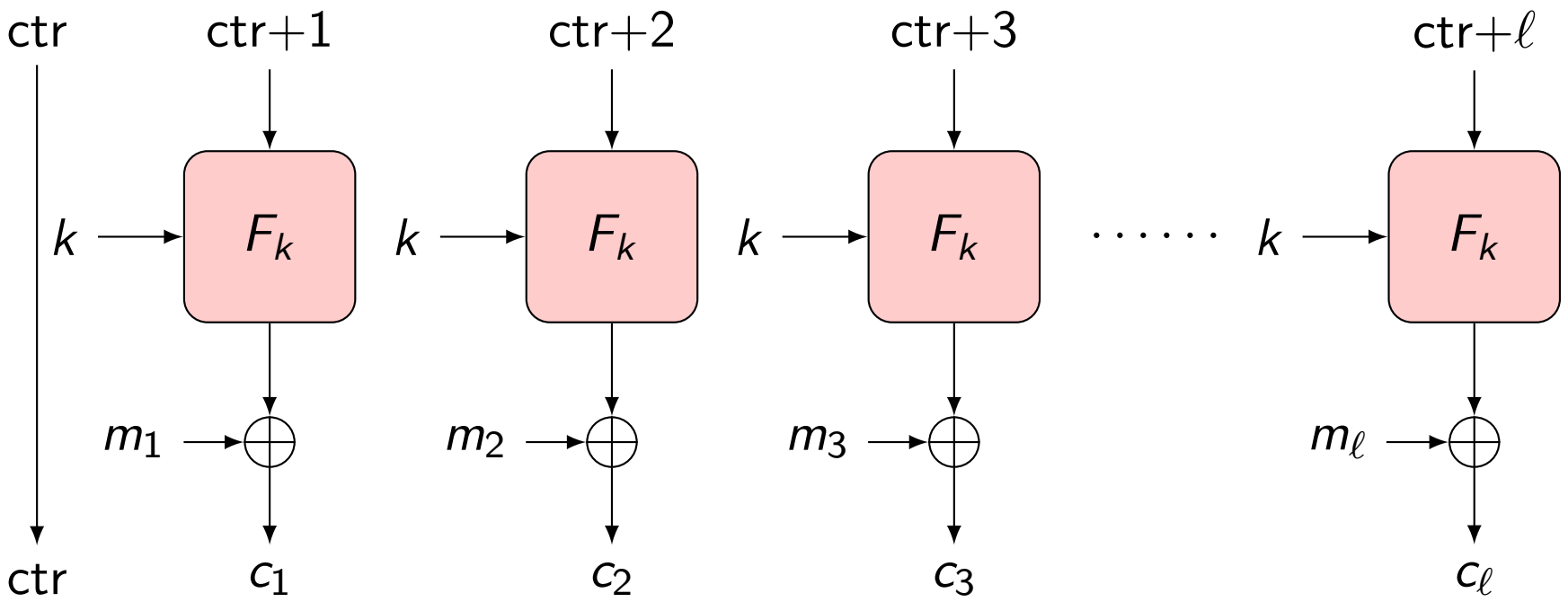
- Ciphertext length: $|c| = |m| + 1$ (adds one block)

CTR Mode Properties



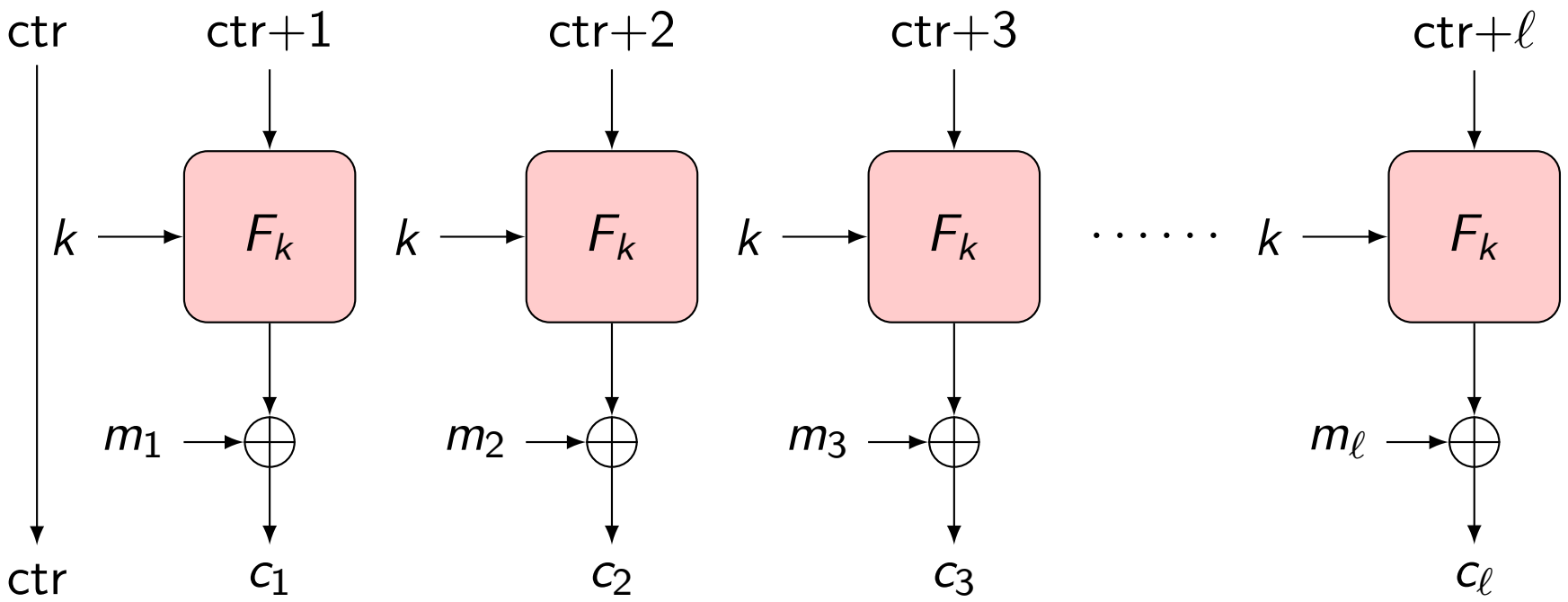
- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation:

CTR Mode Properties



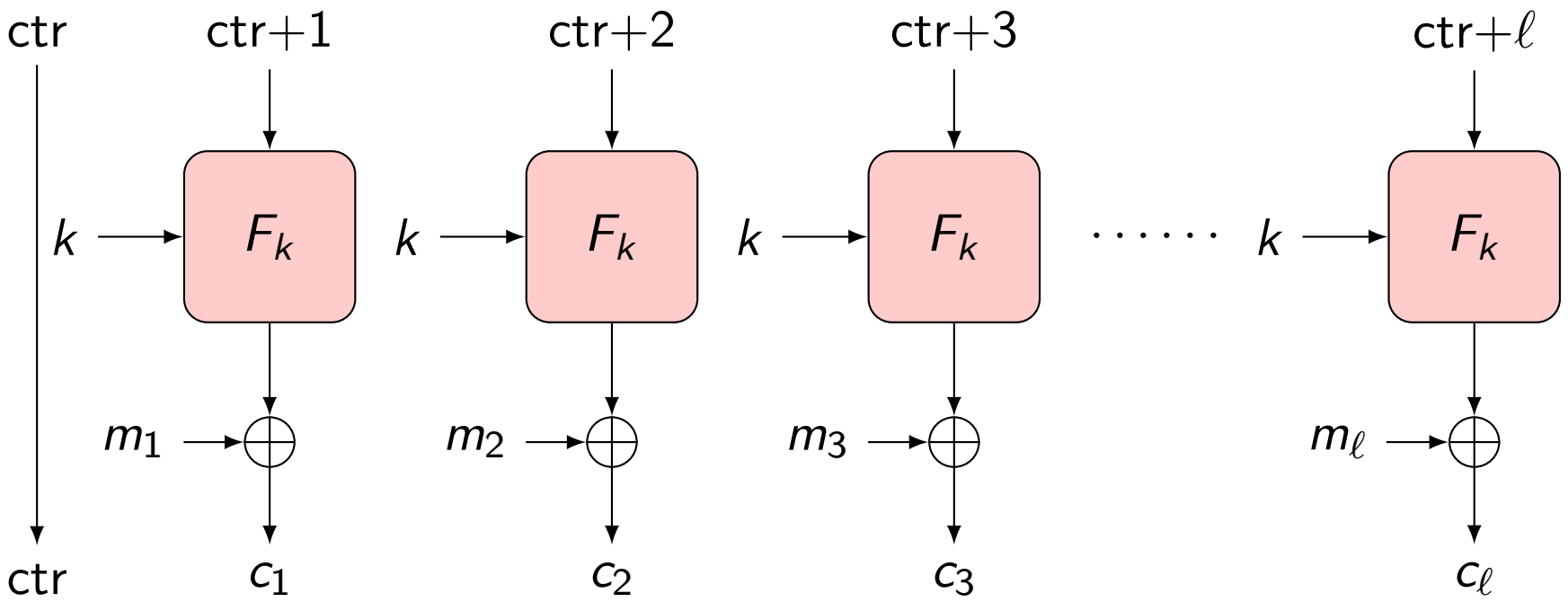
- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads

CTR Mode Properties



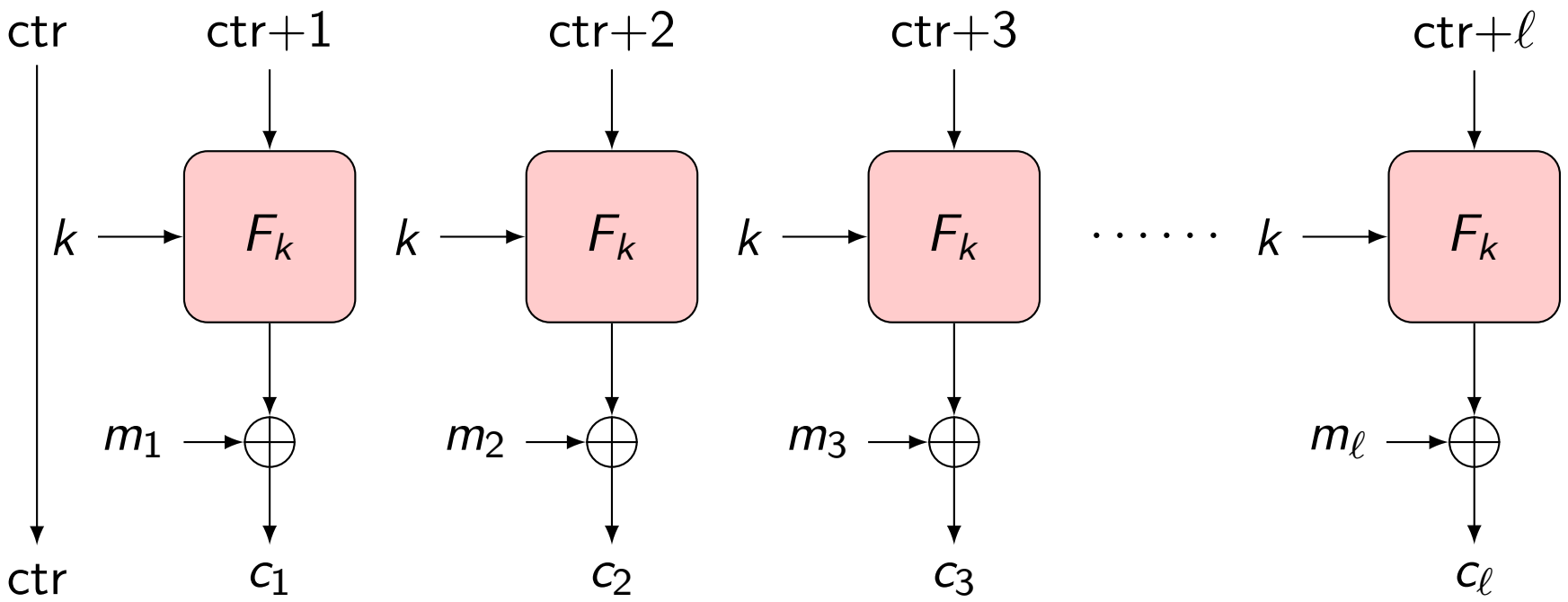
- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads
- Encryption/Decryption parallelism:

CTR Mode Properties



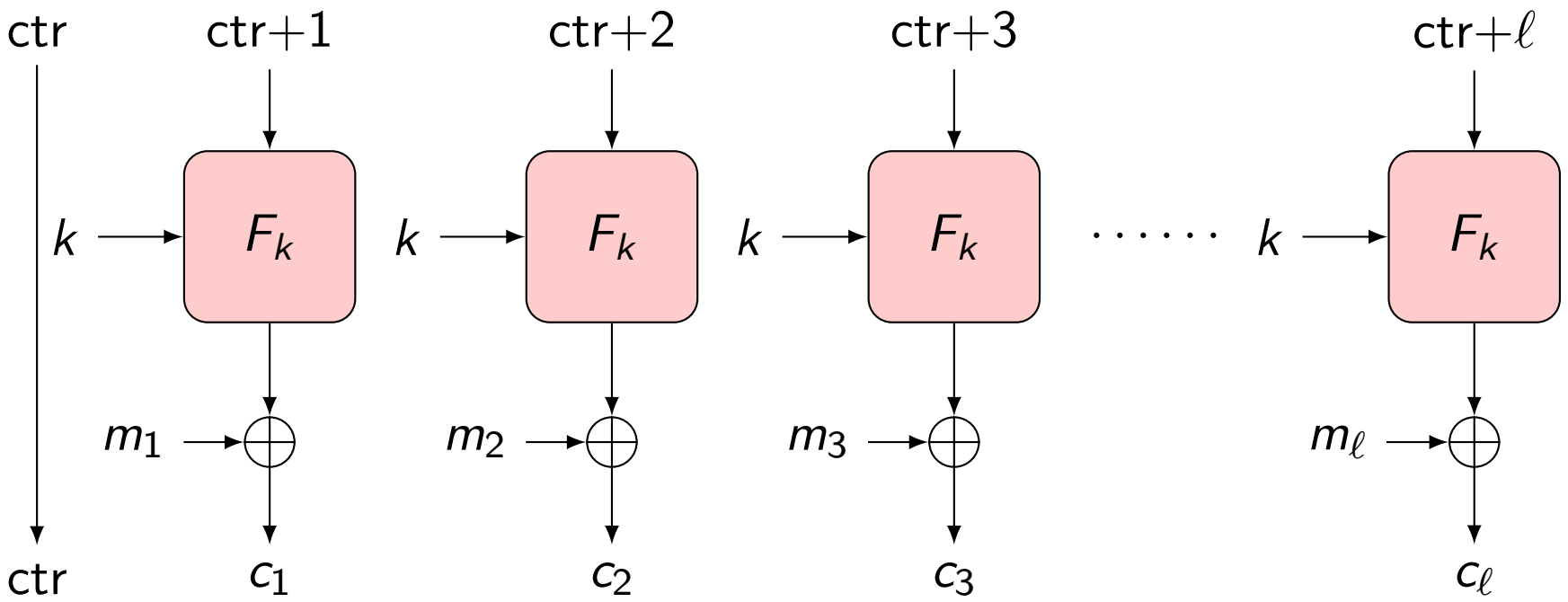
- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads
- Encryption/Decryption parallelism: Can parallelize both encryption and decryption

CTR Mode Properties



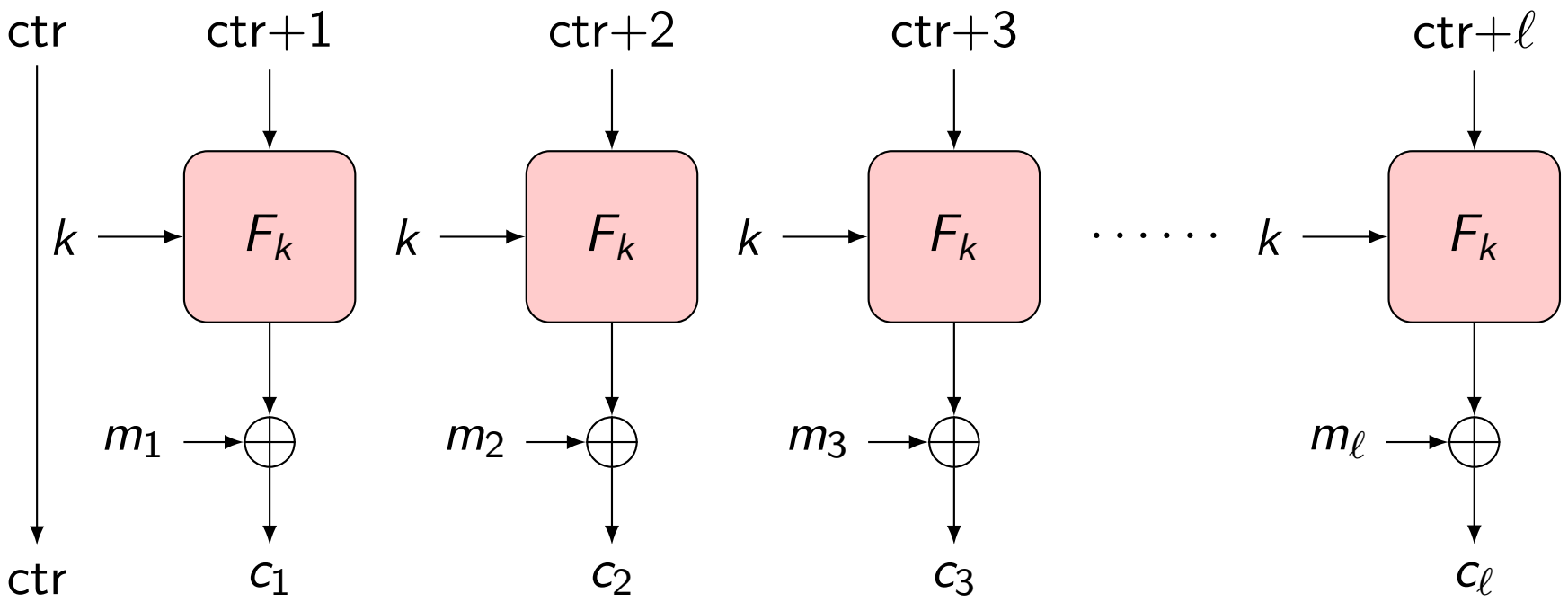
- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads
- Encryption/Decryption parallelism: Can parallelize both encryption and decryption
- PRF Inverse:

CTR Mode Properties



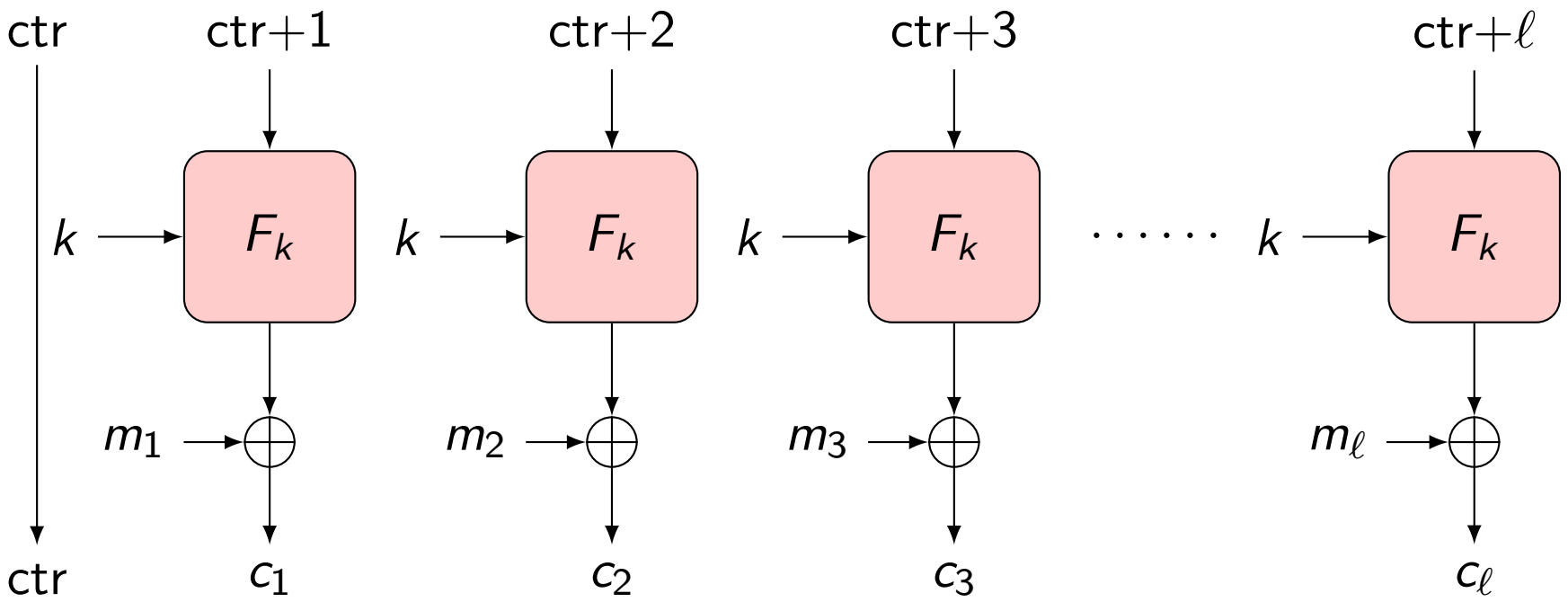
- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads
- Encryption/Decryption parallelism: Can parallelize both encryption and decryption
- PRF Inverse: No

CTR Mode Properties



- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads
- Encryption/Decryption parallelism: Can parallelize both encryption and decryption
- PRF Inverse: No
- Security:

CTR Mode Properties

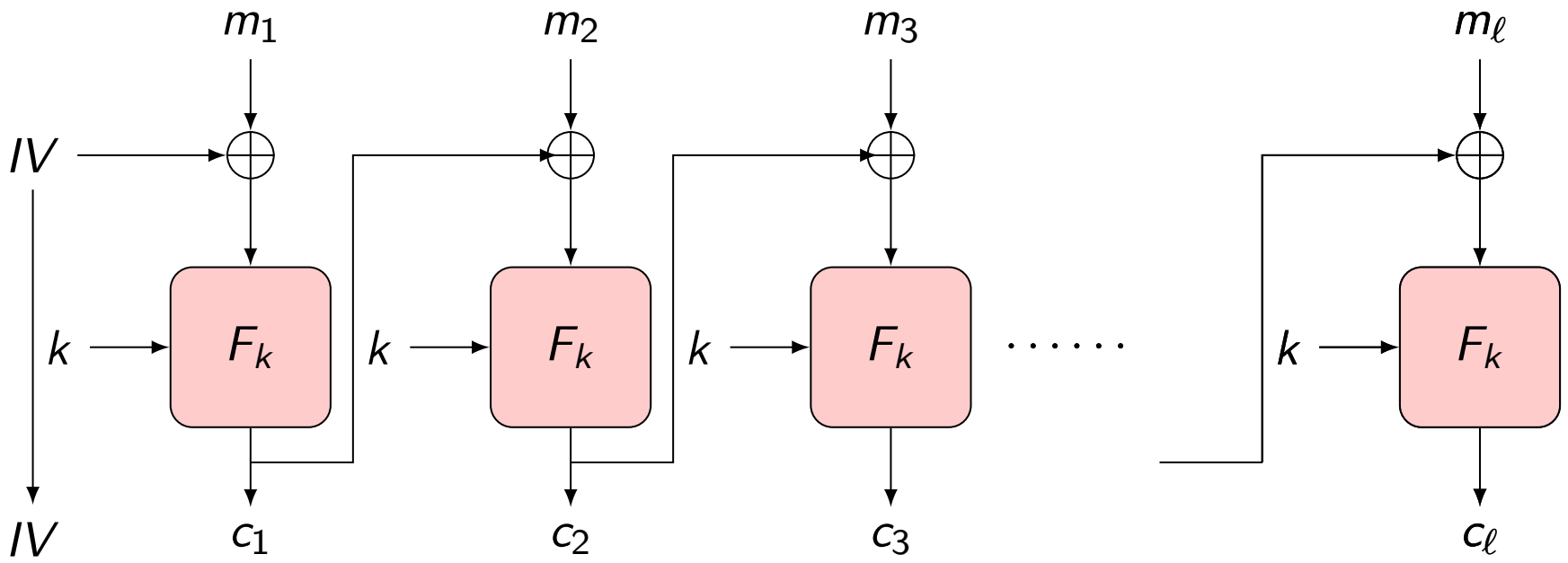


- Ciphertext length: $|c| = |m| + 1$ (adds one block)
- Pre-computation: Can precompute pads
- Encryption/Decryption parallelism: Can parallelize both encryption and decryption
- PRF Inverse: No
- Security: CPA-secure if no $ctr + i$ value is reused under same key

Outline

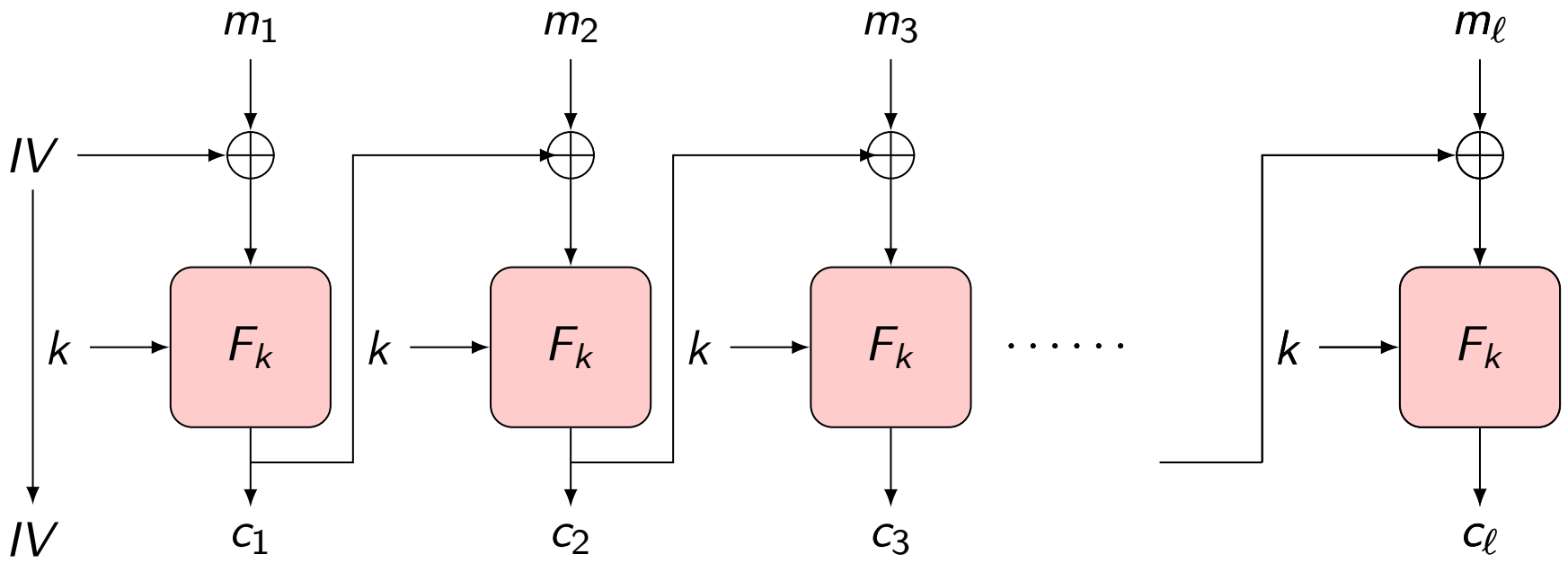
1. Modes of Operation
2. Padding-Oracle Attack on CBC Mode
3. Chosen-Ciphertext Attack (CCA) Security
4. Message Authentication Codes
5. Authenticated Encryption

Let's Recall CBC Mode



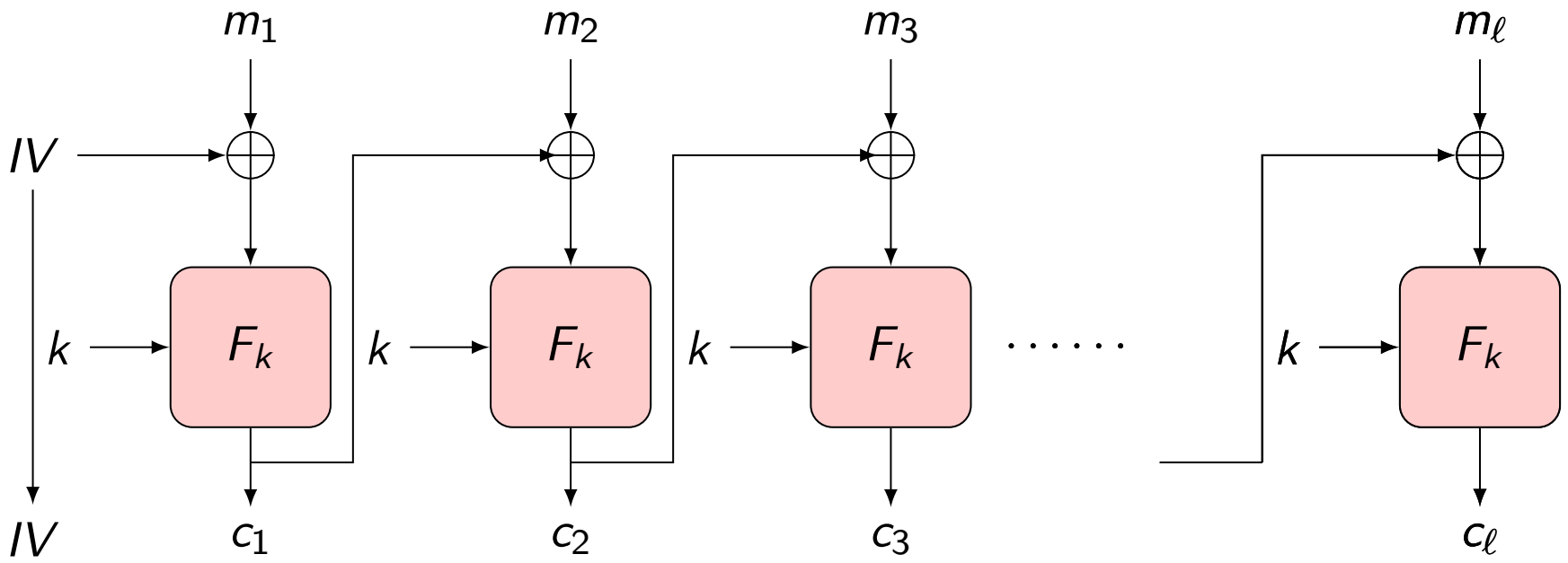
- Every mesage block m_i must be n bits long (e.g., 128 bits for AES)

Let's Recall CBC Mode



- Every mesage block m_i must be n bits long (e.g., 128 bits for AES)
- If the message is not a multiple of n bits, we must pad it to make it so

Let's Recall CBC Mode



- Every mesage block m_i must be n bits long (e.g., 128 bits for AES)
- If the message is not a multiple of n bits, we must pad it to make it so
- But, how do we know where the message ends and the padding begins?

Padding: Make the Last Block Complete

Let L be the block length *in bytes* (AES: $L = 16$). Append b copies of byte b, where

$$b = L - (|m| \bmod L)$$

Here $|m|$ is measured in bytes.

Padding: Make the Last Block Complete

Let L be the block length *in bytes* (AES: $L = 16$). Append b copies of byte b, where

$$b = L - (|m| \bmod L)$$

Here $|m|$ is measured in bytes.

Eight-byte toy blocks; bytes shown in hexadecimal

41 42 43 44 45 $\longrightarrow$ 41 42 43 44 45 03 03 03

Padding: Make the Last Block Complete

Let L be the block length *in bytes* (AES: $L = 16$). Append b copies of byte b, where

$$b = L - (|m| \bmod L)$$

Here $|m|$ is measured in bytes.

Eight-byte toy blocks; bytes shown in hexadecimal

41 42 43 44 45 $\longrightarrow$ 41 42 43 44 45 03 03 03

- Already block-aligned? Append a full block of padding.

Padding: Make the Last Block Complete

Let L be the block length *in bytes* (AES: $L = 16$). Append b copies of byte b, where

$$b = L - (|m| \bmod L)$$

Here $|m|$ is measured in bytes.

Eight-byte toy blocks; bytes shown in hexadecimal

`41 42 43 44 45` $\longrightarrow$ `41 42 43 44 45 03 03 03`

- Already block-aligned? Append a full block of padding.
- To unpad: read the last byte b; check $1 \leq b \leq L$ and that all last b bytes equal b; then remove them.

Padding: Make the Last Block Complete

Let L be the block length *in bytes* (AES: $L = 16$). Append b copies of byte b, where

$$b = L - (|m| \bmod L)$$

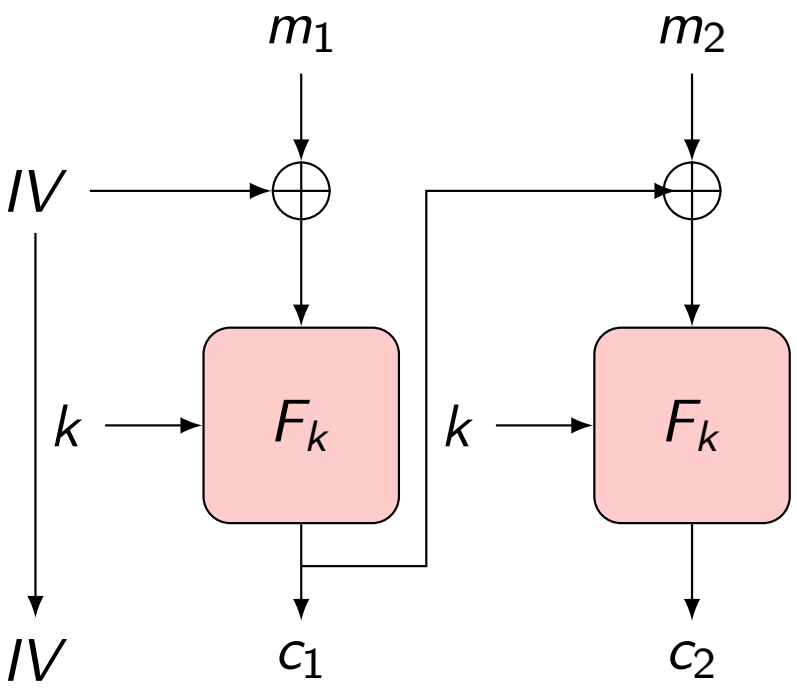
Here $|m|$ is measured in bytes.

Eight-byte toy blocks; bytes shown in hexadecimal

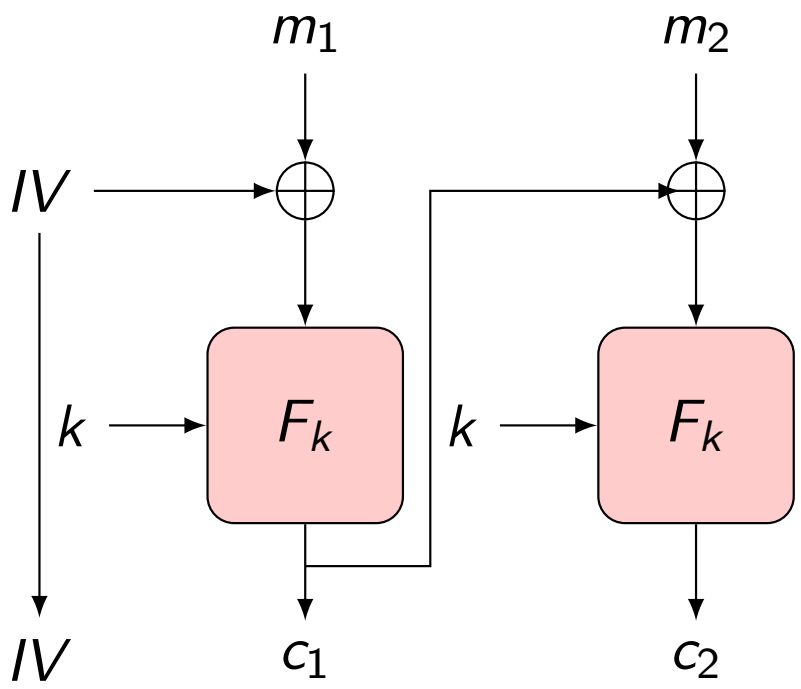
`41 42 43 44 45` $\longrightarrow$ `41 42 43 44 45 03 03 03`

- Already block-aligned? Append a full block of padding.
- To unpad: read the last byte b; check $1 \leq b \leq L$ and that all last b bytes equal b; then remove them.
- BAD PADDING is a common error message returned by decryption when the check fails.

Padding Oracle Attack

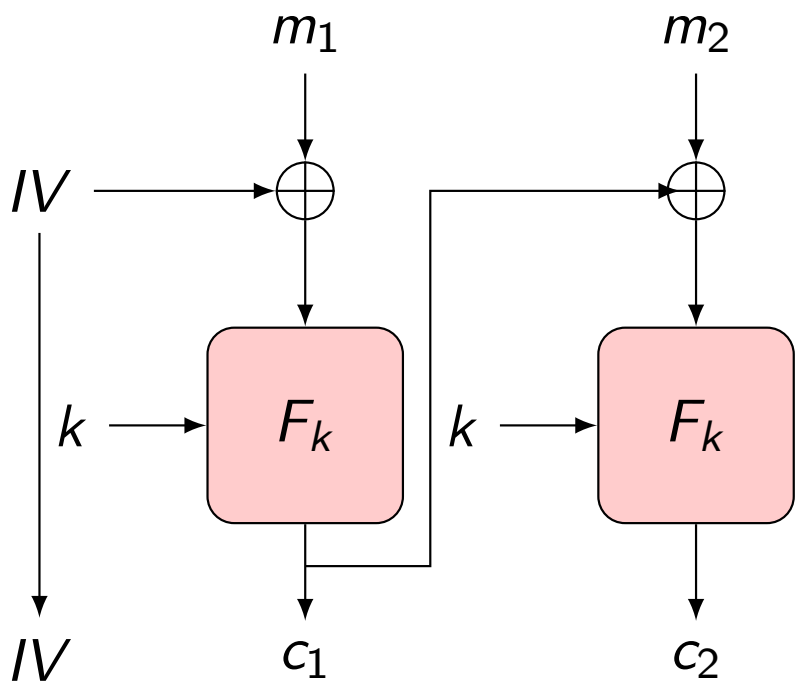


Padding Oracle Attack



- Adversary intercepts (IV, c_1, c_2) from an honest user to a bank
- Adversary can modify it arbitrarily and send (IV, c_1', c_2) to the bank
- Bank decrypts and checks the padding, returning BAD PADDING or OK

Padding Oracle Attack



- Adversary intercepts (IV, c_1, c_2) from an honest user to a bank
- Adversary can modify it arbitrarily and send (IV, c_1', c_2) to the bank
- Bank decrypts and checks the padding, returning BAD PADDING or OK

Your task

Recover plaintext bytes using only this oracle and ciphertext edits.

Quiz: Recover One Unknown Byte

$$m_2 = F_k^{-1}(c_2) \oplus c_1.$$

For this activity, you are given $b = 3$: the final block ends in

$$\ldots \texttt{ ?? 03 03 03}.$$

1. How does changing c_1 change m_2?
2. Construct a query that forces the last four bytes to be 04, exactly when your guess for the unknown byte is correct.
3. How would you repeat this to learn another byte?

Work for 10 minutes

Write an explicit ciphertext edit and explain what acceptance tells you. Finding b and attacking earlier blocks are optional challenges.

Hint: Turn Data into Padding

Suppose the end of m_2 is

$$\ldots \texttt{ ?? 03 03 03.}$$

You want a modified plaintext ending in

$$\ldots \texttt{ 04 04 04 04.}$$

- You know the original values of the last three bytes.
- What XOR changes turn each 03 into 04?
- For the unknown byte, try a guess g. What change makes it 04 exactly when your guess is correct?

Changing m_2

Keep c_2 fixed; choose $c_1' = c_1 \oplus \Delta$.

$$\begin{aligned} m_2' &= F_k^{-1}(c_2) \oplus c_1' \\ &= F_k^{-1}(c_2) \oplus c_1 \oplus \Delta \\ &= m_2 \oplus \Delta. \end{aligned}$$

- We do not know m_2, but we control how it changes.

A Concrete Byte Recovery

Suppose $L = 8$ and $b = 3$. The actual final block is hidden:

$$41\ 42\ 43\ 44\ 45\ 03\ 03\ 03.$$

A Concrete Byte Recovery

Suppose $L = 8$ and $b = 3$. The actual final block is hidden:

$$41\ \ 42\ \ 43\ \ 44\ \ 45\ \ 03\ \ 03\ \ 03.$$

To test guess g, XOR into c_1 the byte mask

$$00\ \ 00\ \ 00\ \ 00\ \ \ (g \oplus 04)\ \ \ 07\ \ 07\ \ 07.$$

For $g = 45$, this is

$$00\ \ 00\ \ 00\ \ 00\ \ 41\ \ 07\ \ 07\ \ 07,$$

A Concrete Byte Recovery

Suppose $L = 8$ and $b = 3$. The actual final block is hidden:

$$41\ \ 42\ \ 43\ \ 44\ \ 45\ \ 03\ \ 03\ \ 03.$$

To test guess g, XOR into c_1 the byte mask

$$00\ \ 00\ \ 00\ \ 00\ \ \ (g \oplus 04)\ \ \ 07\ \ 07\ \ 07.$$

For $g = 45$, this is

$$00\ \ 00\ \ 00\ \ 00\ \ 41\ \ 07\ \ 07\ \ 07,$$

and the modified plaintext ends in

$$41\ \ 42\ \ 43\ \ 44\ \ 04\ \ 04\ \ 04\ \ 04.$$

A Concrete Byte Recovery

Suppose $L = 8$ and $b = 3$. The actual final block is hidden:

$$41\ \ 42\ \ 43\ \ 44\ \ 45\ \ 03\ \ 03\ \ 03.$$

To test guess g, XOR into c_1 the byte mask

$$00\ \ 00\ \ 00\ \ 00\ \ \ (g \oplus 04)\ \ \ 07\ \ 07\ \ 07.$$

For $g = 45$, this is

$$00\ \ 00\ \ 00\ \ 00\ \ 41\ \ 07\ \ 07\ \ 07,$$

and the modified plaintext ends in

$$41\ \ 42\ \ 43\ \ 44\ \ 04\ \ 04\ \ 04\ \ 04.$$

Why the answer is unambiguous

The final byte is fixed to 04. Acceptance identifies the guess. Next force five copies of 05, using the four known suffix bytes, and repeat.

Recover One Unknown Byte

Suppose $b < L$. Set $t = b + 1$ and $j = L - b$. We want the final t bytes to all equal byte t.

1. For every known suffix position $i > j$, set

$$c_1'[i] = c_1[i] \oplus m_2[i] \oplus t.$$

Recover One Unknown Byte

Suppose $b < L$. Set $t = b + 1$ and $j = L - b$. We want the final t bytes to all equal byte t.

1. For every known suffix position $i > j$, set

$$c_1'[i] = c_1[i] \oplus m_2[i] \oplus t.$$

2. For a guess $g \in \{0, \ldots, 255\}$, set

$$c_1'[j] = c_1[j] \oplus g \oplus t.$$

Recover One Unknown Byte

Suppose $b < L$. Set $t = b + 1$ and $j = L - b$. We want the final t bytes to all equal byte t.

1. For every known suffix position $i > j$, set

$$c_1'[i] = c_1[i] \oplus m_2[i] \oplus t.$$

2. For a guess $g \in \{0, \ldots, 255\}$, set

$$c_1'[j] = c_1[j] \oplus g \oplus t.$$

3. Query (IV, c_1', c_2). Valid padding means $g = m_2[j]$.

Recover One Unknown Byte

Suppose $b < L$. Set $t = b + 1$ and $j = L - b$. We want the final t bytes to all equal byte t.

1. For every known suffix position $i > j$, set

$$c_1'[i] = c_1[i] \oplus m_2[i] \oplus t.$$

2. For a guess $g \in \{0, \ldots, 255\}$, set

$$c_1'[j] = c_1[j] \oplus g \oplus t.$$

3. Query (IV, c_1', c_2). Valid padding means $g = m_2[j]$.

Why? The last byte is now t, the known suffix is correct, and

$$m_2'[j] = m_2[j] \oplus g \oplus t = t \quad \Longleftrightarrow \quad g = m_2[j].$$

Completing the Attack

Earlier, we assumed b was known. If it is not, we can learn it first.

Completing the Attack

Earlier, we assumed b was known. If it is not, we can learn it first.

Observation: We know that m_2 ends in $(0x b)$ repeated b times

Completing the Attack

Earlier, we assumed b was known. If it is not, we can learn it first.

Observation: We know that m_2 ends in $(0x\|b)$ repeated b timesStep 1:

Learn size of padding

Completing the Attack

Earlier, we assumed b was known. If it is not, we can learn it first.

Observation: We know that m_2 ends in $(0x b)$ repeated b timesStep 1:

Learn size of padding

- Change 1st Byte of c_1 (thus, also m_2) and see if error occurs
  - Error only occurs if $|pad| = L$

Completing the Attack

Earlier, we assumed b was known. If it is not, we can learn it first.

Observation: We know that m_2 ends in $(0x b)$ repeated b timesStep 1:

Learn size of padding

- Change 1st Byte of c_1 (thus, also m_2) and see if error occurs
  - Error only occurs if $|pad| = L$
- Change Bytes $2, \ldots, L$ until first time we get error, this is first Byte of padding

Step 2: Use the attack we already saw to learn all Bytes of m_2

What Actually Failed?

- CBC can be CPA secure with random IVs.

What Actually Failed?

- CBC can be CPA secure with random IVs.
- CPA security does not protect against decryption feedback.

What Actually Failed?

- CBC can be CPA secure with random IVs.
- CPA security does not protect against decryption feedback.
- CBC lets us modify plaintext in a controlled way.

What Actually Failed?

- CBC can be CPA secure with random IVs.
- CPA security does not protect against decryption feedback.
- CBC lets us modify plaintext in a controlled way.
- BAD PADDING error messages leak information about the plaintext.

What Actually Failed?

- CBC can be CPA secure with random IVs.
- CPA security does not protect against decryption feedback.
- CBC lets us modify plaintext in a controlled way.
- BAD PADDING error messages leak information about the plaintext.

The combination matters

Unauthenticated ciphertexts + controlled modifications + a padding-validity leak can reveal an entire message.

Outline

1. Modes of Operation
2. Padding-Oracle Attack on CBC Mode
3. Chosen-Ciphertext Attack (CCA) Security
4. Message Authentication Codes
5. Authenticated Encryption

From the Attack to a Security Definition

CPA: The attacker can obtain encryptions of chosen messages.

From the Attack to a Security Definition

CPA: The attacker can obtain encryptions of chosen messages.

CCA: The attacker can also obtain decryptions of chosen ciphertexts, before and after receiving the challenge.

From the Attack to a Security Definition

CPA: The attacker can obtain encryptions of chosen messages.

CCA: The attacker can also obtain decryptions of chosen ciphertexts, before and after receiving the challenge.

- The challenge ciphertext itself may not be submitted for decryption.

From the Attack to a Security Definition

CPA: The attacker can obtain encryptions of chosen messages.

CCA: The attacker can also obtain decryptions of chosen ciphertexts, before and after receiving the challenge.

- The challenge ciphertext itself may not be submitted for decryption.
- Modified versions of it *are* allowed.

From the Attack to a Security Definition

CPA: The attacker can obtain encryptions of chosen messages.

CCA: The attacker can also obtain decryptions of chosen ciphertexts, before and after receiving the challenge.

- The challenge ciphertext itself may not be submitted for decryption.
- Modified versions of it *are* allowed.
- The goal is still to hide which challenge message was encrypted.

From the Attack to a Security Definition

CPA: The attacker can obtain encryptions of chosen messages.

CCA: The attacker can also obtain decryptions of chosen ciphertexts, before and after receiving the challenge.

- The challenge ciphertext itself may not be submitted for decryption.
- Modified versions of it *are* allowed.
- The goal is still to hide which challenge message was encrypted.

Think back to the quiz

The padding oracle attack is a CCA attack. The adversary can submit modified ciphertexts for decryption, and the oracle's response reveals information about the challenge plaintext.

The CCA Security Experiment

1. Challenger samples $k \leftarrow \text{Gen}(1^n)$.

The CCA Security Experiment

1. Challenger samples $k \leftarrow \mathrm{Gen}(1^n)$.
2. $\mathcal{A}$ can query $\mathrm{Enc}_k(\cdot)$ and $\mathrm{Dec}_k(\cdot)$, then chooses equal-length messages m_0, m_1.

The CCA Security Experiment

1. Challenger samples $k \leftarrow \mathsf{Gen}(1^n)$.
2. $\mathcal{A}$ can query $\mathsf{Enc}_k(\cdot)$ and $\mathsf{Dec}_k(\cdot)$, then chooses equal-length messages m_0, m_1.
3. Challenger chooses $b \leftarrow \{0,1\}$ and sends

$$c^* \leftarrow \mathsf{Enc}_k(m_b).$$

The CCA Security Experiment

1. Challenger samples $k \leftarrow \mathrm{Gen}(1^n)$.
2. $\mathcal{A}$ can query $\mathrm{Enc}_k(\cdot)$ and $\mathrm{Dec}_k(\cdot)$, then chooses equal-length messages m_0, m_1.
3. Challenger chooses $b \leftarrow \{0,1\}$ and sends

$$c^* \leftarrow \mathrm{Enc}_k(m_b).$$

4. $\mathcal{A}$ continues to query both oracles, but may not ask for $\mathrm{Dec}_k(c^*)$ after receiving the challenge.

The CCA Security Experiment

1. Challenger samples $k \leftarrow \mathrm{Gen}(1^n)$.
2. $\mathcal{A}$ can query $\mathrm{Enc}_k(\cdot)$ and $\mathrm{Dec}_k(\cdot)$, then chooses equal-length messages m_0, m_1.
3. Challenger chooses $b \leftarrow \{0,1\}$ and sends

$$c^* \leftarrow \mathrm{Enc}_k(m_b).$$

4. $\mathcal{A}$ continues to query both oracles, but may not ask for $\mathrm{Dec}_k(c^*)$ after receiving the challenge.
5. $\mathcal{A}$ outputs b'; it wins if $b' = b$.

The CCA Security Experiment

1. Challenger samples $k \leftarrow \mathrm{Gen}(1^n)$.
2. $\mathcal{A}$ can query $\mathrm{Enc}_k(\cdot)$ and $\mathrm{Dec}_k(\cdot)$, then chooses equal-length messages m_0, m_1.
3. Challenger chooses $b \leftarrow \{0,1\}$ and sends

$$c^* \leftarrow \mathrm{Enc}_k(m_b).$$

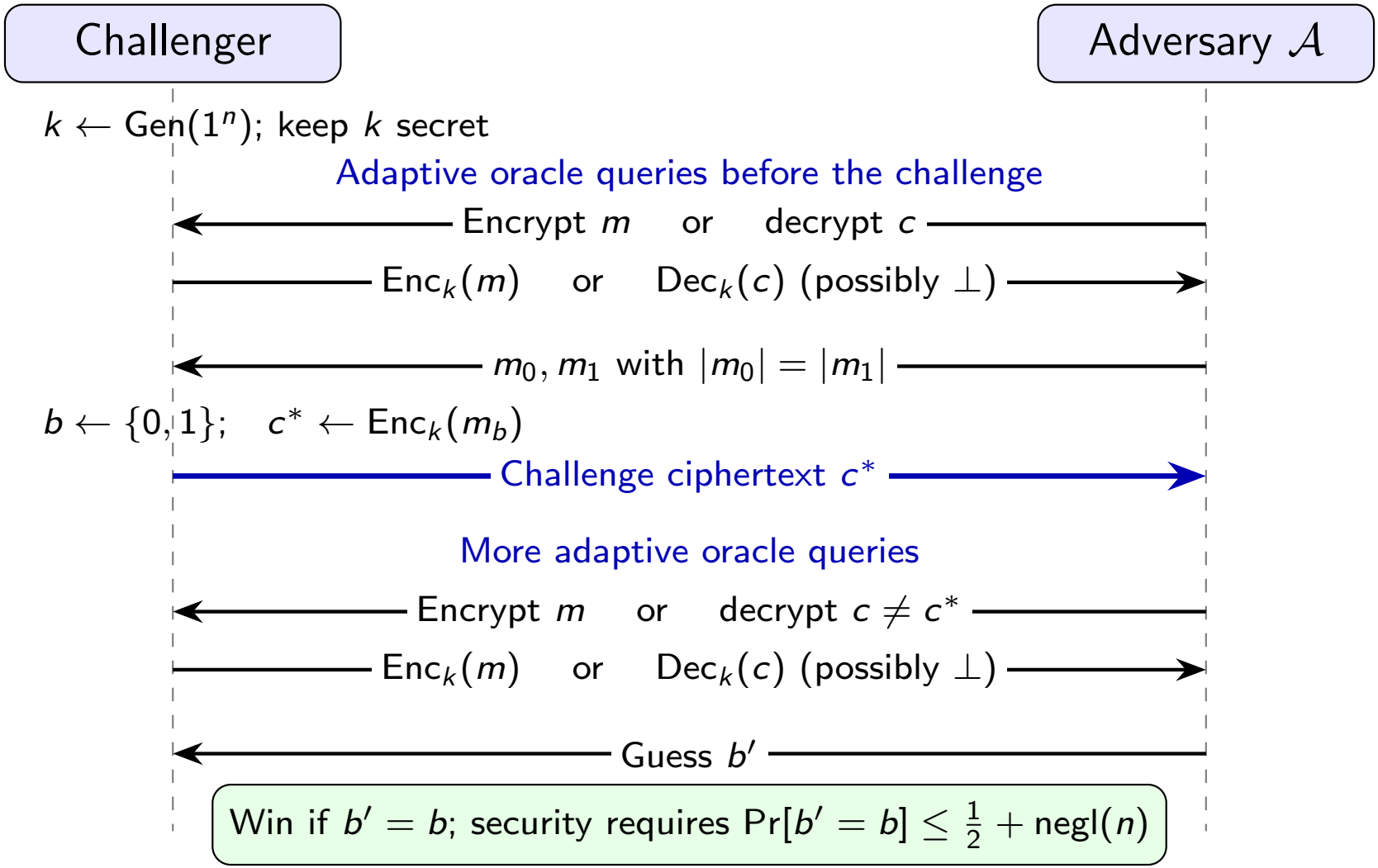
4. $\mathcal{A}$ continues to query both oracles, but may not ask for $\mathrm{Dec}_k(c^*)$ after receiving the challenge.
5. $\mathcal{A}$ outputs b'; it wins if $b' = b$.

CCA security (adaptive chosen-ciphertext security)

For every PPT adversary $\mathcal{A}$,

$$\Pr[\mathrm{PrivK}^{cca}_{\mathcal{A},\Pi}(n) = 1] \leq \tfrac{1}{2} + \mathrm{negl}(n).$$

The CCA Experiment: Challenger and Adversary



Is PRF+OTP CCA-Secure?

Recall $\mathrm{Enc}_k(m) = (r, F_k(r) \oplus m)$.

1. Receive challenge $c^* = (r^*, s^*)$.

Is PRF+OTP CCA-Secure?

Recall $\mathrm{Enc}_k(m) = (r, F_k(r) \oplus m)$.

1. Receive challenge $c^* = (r^*, s^*)$.

How do we fix this?

Is PRF+OTP CCA-Secure?

Recall $\mathrm{Enc}_k(m) = (r, F_k(r) \oplus m)$.

1. Receive challenge $c^* = (r^*, s^*)$.

How do we fix this?

- We need to add integrity protection to the ciphertexts.

Is PRF+OTP CCA-Secure?

Recall $\mathrm{Enc}_k(m) = (r, F_k(r) \oplus m)$.

1. Receive challenge $c^* = (r^*, s^*)$.

How do we fix this?

- We need to add integrity protection to the ciphertexts.
- This can be done with a MAC, which we will discuss next.

Outline

1. Modes of Operation
2. Padding-Oracle Attack on CBC Mode
3. Chosen-Ciphertext Attack (CCA) Security
4. Message Authentication Codes
5. Authenticated Encryption

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough
  - Possible to change content of encrypted messages
  - Often want to know who sent the message

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough
  - Possible to change content of encrypted messages
  - Often want to know who sent the message
- Need to guarantee *integrity* of messages

Example:

- Consider a transaction sent from a client to a bank
- The bank needs to be sure that:

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough
  - Possible to change content of encrypted messages
  - Often want to know who sent the message
- Need to guarantee *integrity* of messages

Example:

- Consider a transaction sent from a client to a bank
- The bank needs to be sure that:
  1. The transaction was sent by the user holding the account

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough
  - Possible to change content of encrypted messages
  - Often want to know who sent the message
- Need to guarantee *integrity* of messages

Example:

- Consider a transaction sent from a client to a bank
- The bank needs to be sure that:
  1. The transaction was sent by the user holding the account
  2. The amounts in the transaction were not modified in transit

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough
  - Possible to change content of encrypted messages
  - Often want to know who sent the message
- Need to guarantee *integrity* of messages

Example:

- Consider a transaction sent from a client to a bank
- The bank needs to be sure that:
  1. The transaction was sent by the user holding the account
  2. The amounts in the transaction were not modified in transit
- Integrity guarantees are needed even if transaction is public

Secrecy vs. Integrity

- Encryption hides content of message – confidentiality
- Often confidentiality is not enough
  - Possible to change content of encrypted messages
  - Often want to know who sent the message
- Need to guarantee *integrity* of messages

Example:

- Consider a transaction sent from a client to a bank
- The bank needs to be sure that:
  1. The transaction was sent by the user holding the account
  2. The amounts in the transaction were not modified in transit
- Integrity guarantees are needed even if transaction is public

We Need Integrity

Confidentiality alone is insufficient to secure information

What Should Happen to a Modified Ciphertext?

What Should Happen to a Modified Ciphertext?

Reject it before releasing decryption information.

What Should Happen to a Modified Ciphertext?

Reject it before releasing decryption information.

- We need a way to detect unauthorized modifications.

What Should Happen to a Modified Ciphertext?

Reject it before releasing decryption information.

- We need a way to detect unauthorized modifications.
- An ordinary checksum is insufficient: an attacker can recompute it.

What Should Happen to a Modified Ciphertext?

Reject it before releasing decryption information.

- We need a way to detect unauthorized modifications.
- An ordinary checksum is insufficient: an attacker can recompute it.
- A secret key can make a valid authentication tag hard to forge.

What Should Happen to a Modified Ciphertext?

Reject it before releasing decryption information.

- We need a way to detect unauthorized modifications.
- An ordinary checksum is insufficient: an attacker can recompute it.
- A secret key can make a valid authentication tag hard to forge.

Message authentication code (MAC)

A shared-key mechanism for authenticating a message.

MAC Functionality

A Message Authentication Scheme (MAC) consists of:

- $\text{Gen}(1^n)$: Outputs key k with $|k| \geq n$ (usually $k \leftarrow \{0,1\}^n$)
- $\text{Mac}_k(m)$: Outputs a tag $t \leftarrow \text{Mac}_k(m)$
- $\text{Verify}_k(m, t)$: Outputs 1 if t is a valid tag for m, and 0 otherwise

MAC Functionality

A Message Authentication Scheme (MAC) consists of:

- $\text{Gen}(1^n)$: Outputs key k with $|k| \geq n$ (usually $k \leftarrow \{0,1\}^n$)
- $\text{Mac}_k(m)$: Outputs a tag $t \leftarrow \text{Mac}_k(m)$
- $\text{Verify}_k(m, t)$: Outputs 1 if t is a valid tag for m, and 0 otherwise

Correctness

For all k output by Gen and all messages m, $\text{Verify}_k(m, \text{Mac}_k(m)) = 1$

MAC Functionality

A Message Authentication Scheme (MAC) consists of:

- $\text{Gen}(1^n)$: Outputs key k with $|k| \geq n$ (usually $k \leftarrow \{0,1\}^n$)
- $\text{Mac}_k(m)$: Outputs a tag $t \leftarrow \text{Mac}_k(m)$
- $\text{Verify}_k(m, t)$: Outputs 1 if t is a valid tag for m, and 0 otherwise

Correctness

For all k output by Gen and all messages m, $\text{Verify}_k(m, \text{Mac}_k(m)) = 1$

Canonical Verify

If Mac is deterministic, Verify can compute $\text{Mac}_k(m)$, check equality to t.

MAC Unforgeability

Let $\Pi = (\mathsf{Gen}, \mathsf{Mac}, \mathsf{Verify})$ be a MAC. Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\mathsf{MacForge}_{\mathcal{A},\Pi}(n)$

MAC Unforgeability

Let $\Pi = (\text{Gen}, \text{Mac}, \text{Verify})$ be a MAC. Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{MacForge}_{\mathcal{A},\Pi}(n)$

- The challenger chooses $k \leftarrow \text{Gen}(1^n)$, and gives $\mathcal{A}$ an oracle $\text{Mac}_k(\cdot)$

MAC Unforgeability

Let $\Pi = (\text{Gen}, \text{Mac}, \text{Verify})$ be a MAC. Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{MacForge}_{\mathcal{A},\Pi}(n)$

- The challenger chooses $k \leftarrow \text{Gen}(1^n)$, and gives $\mathcal{A}$ an oracle $\text{Mac}_k(\cdot)$
- $\mathcal{A}^{\text{Mac}_k(\cdot)}(1^n)$ outputs (m, t) (let Q be set of Mac queries made by $\mathcal{A}$)

MAC Unforgeability

Let $\Pi = (\text{Gen}, \text{Mac}, \text{Verify})$ be a MAC. Consider the following game between an adversary $\mathcal{A}$ and a challenger:

$\text{MacForge}_{\mathcal{A},\Pi}(n)$

- The challenger chooses $k \leftarrow \text{Gen}(1^n)$, and gives $\mathcal{A}$ an oracle $\text{Mac}_k(\cdot)$
- $\mathcal{A}^{\text{Mac}_k(\cdot)}(1^n)$ outputs (m, t) (let Q be set of Mac queries made by $\mathcal{A}$)
- We say that $\text{MacForge}_{\mathcal{A},\Pi}(n) = 1$ (i.e., $\mathcal{A}$ wins) if
  - $\text{Verify}_k(m, t) = 1$
  - $m \notin Q$

MAC Unforgeability

Let $\Pi = (\text{Gen}, \text{Mac}, \text{Verify})$ be a MAC. Consider the following game between an adversary $\mathcal{A}$ and a challenger:

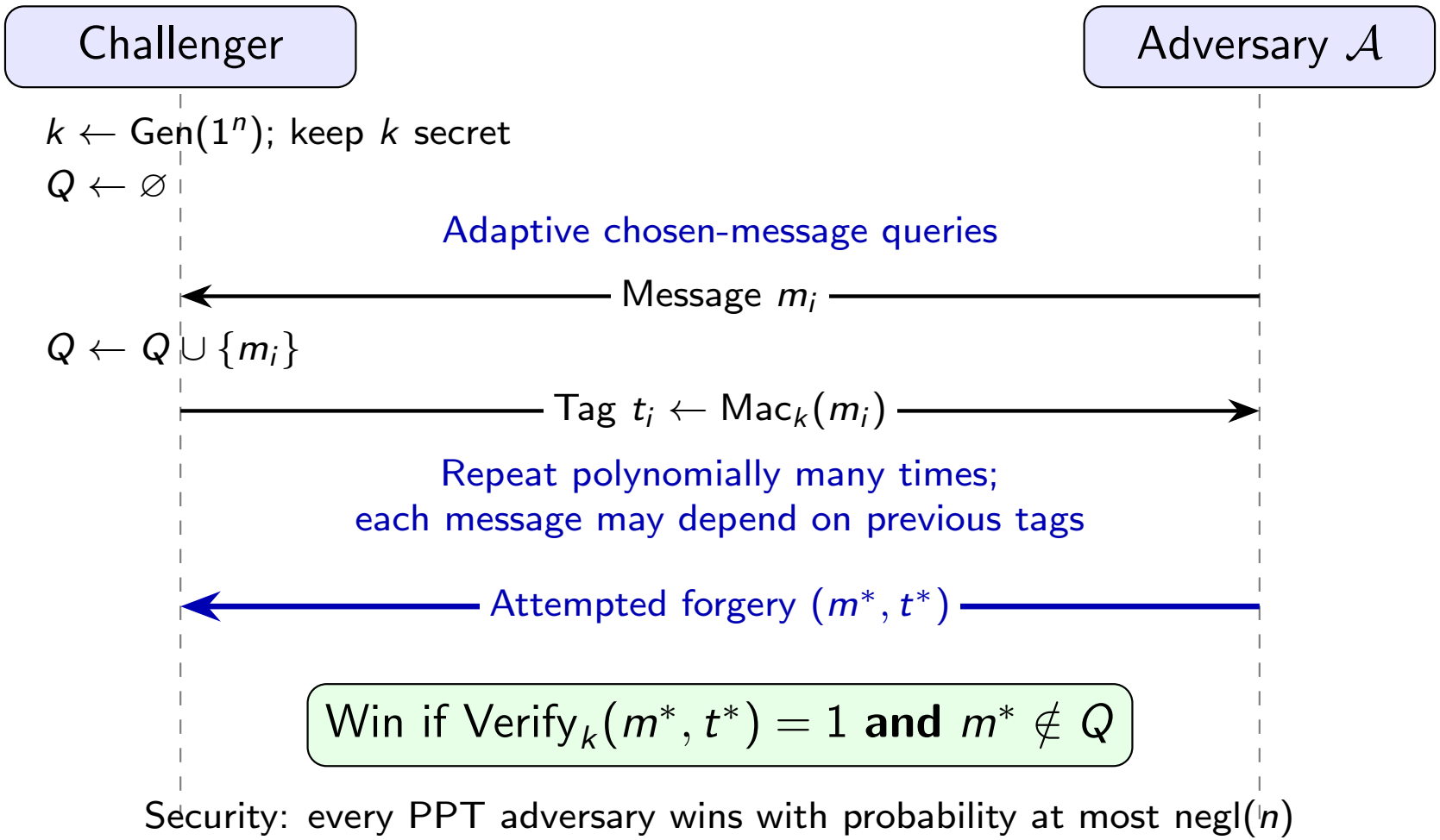
$\text{MacForge}_{\mathcal{A},\Pi}(n)$

- The challenger chooses $k \leftarrow \text{Gen}(1^n)$, and gives $\mathcal{A}$ an oracle $\text{Mac}_k(\cdot)$
- $\mathcal{A}^{\text{Mac}_k(\cdot)}(1^n)$ outputs (m, t) (let Q be set of Mac queries made by $\mathcal{A}$)
- We say that $\text{MacForge}_{\mathcal{A},\Pi}(n) = 1$ (i.e., $\mathcal{A}$ wins) if
  - $\text{Verify}_k(m, t) = 1$
  - $m \notin Q$

Definition: A MAC $\Pi = (\text{Gen}, \text{Mac}, \text{Verify})$ is *unforgeable* if for all PPT $\mathcal{A}$ it holds that

$$\Pr[\text{MacForge}_{\mathcal{A},\Pi}(n) = 1] \leq \text{negl}(n)$$

The MAC Security Game: Challenger and Adversary



Security: every PPT adversary wins with probability at most $\mathrm{negl}(n)$

Observations and Variants

Observations and Variants

- A MAC provides integrity, not secrecy: the message may be public.

Observations and Variants

- A MAC provides integrity, not secrecy: the message may be public.
- A previously authenticated message can be replayed. Preventing replay requires protocol state, such as sequence numbers.

Observations and Variants

- A MAC provides integrity, not secrecy: the message may be public.
- A previously authenticated message can be replayed. Preventing replay requires protocol state, such as sequence numbers.

Strong MAC

The attacker wins by producing any valid pair (m, t) that was not returned by the tagging oracle, even if m was queried before.

Observations and Variants

- A MAC provides integrity, not secrecy: the message may be public.
- A previously authenticated message can be replayed. Preventing replay requires protocol state, such as sequence numbers.

Strong MAC

The attacker wins by producing any valid pair (m, t) that was not returned by the tagging oracle, even if m was queried before.

A deterministic unforgeable MAC with canonical verification is also strongly unforgeable: each message has exactly one valid tag.

Unforgeable MAC from a PRF

PRF-based MAC (Fixed Length MAC)

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$

Unforgeable MAC from a PRF

PRF-based MAC (Fixed Length MAC)

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$
- $\text{Mac}_k(m)$: Given $m \in \{0,1\}^n$, compute $t = F_k(m)$

Unforgeable MAC from a PRF

PRF-based MAC (Fixed Length MAC)

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$
- $\text{Mac}_k(m)$: Given $m \in \{0,1\}^n$, compute $t = F_k(m)$
- $\text{Verify}_k(m,t)$: Compute $t' = F_k(m)$ output 1 iff $t = t'$

Unforgeable MAC from a PRF

PRF-based MAC (Fixed Length MAC)

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$
- $\text{Mac}_k(m)$: Given $m \in \{0,1\}^n$, compute $t = F_k(m)$
- $\text{Verify}_k(m,t)$: Compute $t' = F_k(m)$ output 1 iff $t = t'$

Observations:

- Since $F_k(m)$ is unpredictable (without knowing k), $\mathcal{A}$ can't find tag for new message

Unforgeable MAC from a PRF

PRF-based MAC (Fixed Length MAC)

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$
- $\text{Mac}_k(m)$: Given $m \in \{0,1\}^n$, compute $t = F_k(m)$
- $\text{Verify}_k(m,t)$: Compute $t' = F_k(m)$ output 1 iff $t = t'$

Observations:

- Since $F_k(m)$ is unpredictable (without knowing k), $\mathcal{A}$ can't find tag for new message
- We use canonical verify, so this is strong MAC

Unforgeable MAC from a PRF

PRF-based MAC (Fixed Length MAC)

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$
- $\text{Mac}_k(m)$: Given $m \in \{0,1\}^n$, compute $t = F_k(m)$
- $\text{Verify}_k(m,t)$: Compute $t' = F_k(m)$ output 1 iff $t = t'$

Observations:

- Since $F_k(m)$ is unpredictable (without knowing k), $\mathcal{A}$ can't find tag for new message
- We use canonical verify, so this is strong MAC
- Proof idea: replace F_k by a random function f. At a new message, its tag is independent and uniform, so a guess succeeds with probability 2^{-n}.

Outline

1. Modes of Operation
2. Padding-Oracle Attack on CBC Mode
3. Chosen-Ciphertext Attack (CCA) Security
4. Message Authentication Codes
5. Authenticated Encryption

What Do We Want from a Secure Channel?

What Do We Want from a Secure Channel?

- Confidentiality, even with access to decryption of other ciphertexts.

What Do We Want from a Secure Channel?

- Confidentiality, even with access to decryption of other ciphertexts.
- Detection of modified ciphertexts.

What Do We Want from a Secure Channel?

- Confidentiality, even with access to decryption of other ciphertexts.
- Detection of modified ciphertexts.
- No new valid ciphertexts created by an attacker.

What Do We Want from a Secure Channel?

- Confidentiality, even with access to decryption of other ciphertexts.
- Detection of modified ciphertexts.
- No new valid ciphertexts created by an attacker.

CCA security is a confidentiality guarantee

It hides the challenge message. By itself, it does not rule out producing valid ciphertexts unrelated to that challenge.

What Do We Want from a Secure Channel?

- Confidentiality, even with access to decryption of other ciphertexts.
- Detection of modified ciphertexts.
- No new valid ciphertexts created by an attacker.

CCA security is a confidentiality guarantee

It hides the challenge message. By itself, it does not rule out producing valid ciphertexts unrelated to that challenge.

Authenticated encryption combines confidentiality with ciphertext integrity.

Ciphertext Integrity

Ciphertext Integrity

1. Challenger generates k and offers an encryption oracle.

Ciphertext Integrity

1. Challenger generates k and offers an encryption oracle.
2. Record every ciphertext returned by the oracle in a set Q_C.

Ciphertext Integrity

1. Challenger generates k and offers an encryption oracle.
2. Record every ciphertext returned by the oracle in a set Q_C.
3. The attacker outputs a ciphertext C^*.

Ciphertext Integrity

1. Challenger generates k and offers an encryption oracle.
2. Record every ciphertext returned by the oracle in a set Q_C.
3. The attacker outputs a ciphertext C^*.

A successful forgery

$$C^* \notin Q_C \quad \text{and} \quad \mathrm{Dec}_k(C^*) \neq \perp.$$

The attacker makes a new ciphertext that the receiver accepts.

Ciphertext Integrity

1. Challenger generates k and offers an encryption oracle.
2. Record every ciphertext returned by the oracle in a set Q_C.
3. The attacker outputs a ciphertext C^*.

A successful forgery

$$C^* \notin Q_C \quad \text{and} \quad \mathrm{Dec}_k(C^*) \neq \perp.$$

The attacker makes a new ciphertext that the receiver accepts.

- The decrypted message need not be new.
- Replaying a previously returned ciphertext is excluded from this game; replay protection is a separate protocol requirement.

Encrypt, Then Authenticate

Use independent encryption and MAC keys k_E, k_M.

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

Send $C = (c, t)$, including the IV or nonce in the authenticated c.

Encrypt, Then Authenticate

Use independent encryption and MAC keys k_E, k_M.

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

Send $C = (c, t)$, including the IV or nonce in the authenticated c.

1. Verify t on the entire ciphertext c.

Encrypt, Then Authenticate

Use independent encryption and MAC keys k_E, k_M.

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

Send $C = (c, t)$, including the IV or nonce in the authenticated c.

1. Verify t on the entire ciphertext c.
2. If verification fails, return $\perp$.

Encrypt, Then Authenticate

Use independent encryption and MAC keys k_E, k_M.

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

Send $C = (c, t)$, including the IV or nonce in the authenticated c.

1. Verify t on the entire ciphertext c.
2. If verification fails, return $\perp$.
3. Only after successful verification, decrypt and unpad.

Encrypt, Then Authenticate

Use independent encryption and MAC keys k_E, k_M.

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

Send $C = (c, t)$, including the IV or nonce in the authenticated c.

1. Verify t on the entire ciphertext c.
2. If verification fails, return $\perp$.
3. Only after successful verification, decrypt and unpad.

Composition theorem

CPA-secure encryption + a strongly unforgeable MAC gives authenticated encryption: CCA confidentiality and ciphertext integrity.

Encrypt, Then Authenticate

Use independent encryption and MAC keys k_E, k_M.

$$c \leftarrow \mathrm{Enc}_{k_E}(m), \qquad t \leftarrow \mathrm{Mac}_{k_M}(c).$$

Send $C = (c, t)$, including the IV or nonce in the authenticated c.

1. Verify t on the entire ciphertext c.
2. If verification fails, return $\perp$.
3. Only after successful verification, decrypt and unpad.

Composition theorem

CPA-secure encryption + a strongly unforgeable MAC gives authenticated encryption: CCA confidentiality and ciphertext integrity.

The MAC must support the full ciphertext length. Next lecture: Hash-and-MAC.

Why Encrypt-Then-Authenticate Works

1. Any new accepted pair (c, t) is a strong MAC forgery. This occurs only with negligible probability.

Why Encrypt-Then-Authenticate Works

1. Any new accepted pair (c, t) is a strong MAC forgery. This occurs only with negligible probability.
2. Except for that event, simulate decryption queries:
   - Previously returned encryption: return its recorded plaintext.
   - New ciphertext/tag pair: return $\perp$.
   - Exact challenge pair: forbidden by the CCA game.

Why Encrypt-Then-Authenticate Works

1. Any new accepted pair (c, t) is a strong MAC forgery. This occurs only with negligible probability.
2. Except for that event, simulate decryption queries:
   - Previously returned encryption: return its recorded plaintext.
   - New ciphertext/tag pair: return $\perp$.
   - Exact challenge pair: forbidden by the CCA game.
3. The attacker gains no new decryption information. Its remaining ability to distinguish messages would break CPA security.

Why Encrypt-Then-Authenticate Works

1. Any new accepted pair (c, t) is a strong MAC forgery. This occurs only with negligible probability.
2. Except for that event, simulate decryption queries:
   - Previously returned encryption: return its recorded plaintext.
   - New ciphertext/tag pair: return $\perp$.
   - Exact challenge pair: forbidden by the CCA game.
3. The attacker gains no new decryption information. Its remaining ability to distinguish messages would break CPA security.

Back to the padding oracle

A modified ciphertext is rejected before its padding is inspected, unless the attacker can forge a tag.

Why Not Encrypt and Authenticate Separately?

Consider sending $(\mathrm{Enc}_{k_E}(m), \mathrm{Mac}_{k_M}(m))$.

Why Not Encrypt and Authenticate Separately?

Consider sending $(\mathrm{Enc}_{k_E}(m), \mathrm{Mac}_{k_M}(m))$.

- MAC security does not promise that a tag hides the message.

Why Not Encrypt and Authenticate Separately?

Consider sending $(\mathrm{Enc}_{k_E}(m), \mathrm{Mac}_{k_M}(m))$.

- MAC security does not promise that a tag hides the message.
- If $t = \mathrm{Mac}_k(m)$ is secure, then $t' = (t, m)$ is also an unforgeable tag: forging it still requires forging t.
- But sending t' openly reveals m!

Why Not Encrypt and Authenticate Separately?

Consider sending $(\mathrm{Enc}_{k_E}(m), \mathrm{Mac}_{k_M}(m))$.

- MAC security does not promise that a tag hides the message.
- If $t = \mathrm{Mac}_k(m)$ is secure, then $t' = (t, m)$ is also an unforgeable tag: forging it still requires forging t.
- But sending t' openly reveals m!

Lesson

Secure components do not make every composition secure.
Encrypt-then-authenticate tags the ciphertext, whose contents are already hidden.

From Authenticated Encryption to a Protocol

- Use independent encryption and authentication keys.
- Authenticate the IV/nonce and all ciphertext blocks.
- Nonce requirements still apply; authentication does not repair reused CTR pads.
- To prevent replay, authenticate a sequence number and check it against receiver state.

From Authenticated Encryption to a Protocol

- Use independent encryption and authentication keys.
- Authenticate the IV/nonce and all ciphertext blocks.
- Nonce requirements still apply; authentication does not repair reused CTR pads.
- To prevent replay, authenticate a sequence number and check it against receiver state.

AEAD: authenticated encryption with associated data

Some information, such as a header, should be authenticated but remain public. Associated data is bound to the ciphertext without being encrypted.

From Authenticated Encryption to a Protocol

- Use independent encryption and authentication keys.
- Authenticate the IV/nonce and all ciphertext blocks.
- Nonce requirements still apply; authentication does not repair reused CTR pads.
- To prevent replay, authenticate a sequence number and check it against receiver state.

AEAD: authenticated encryption with associated data

Some information, such as a header, should be authenticated but remain public. Associated data is bound to the ciphertext without being encrypted.

Next week: use hashes to authenticate long messages and large collections of files.